



CKA Exercises & Mock Exams — Questions and Solutions

CKA Q and A

All practice questions (31 exercises + Mock Exams 01 & 02) with solutions, consolidated in one file.

Mock Exam 01

Q1. Pod Deployment Troubleshooting

A pod named `app-frontend` in namespace `production` is stuck in `CrashLoopBackOff`. The pod's container is looking for a configuration file at `/etc/app/config.yaml`, but this file doesn't exist in the image.

You need to: 1. Create a ConfigMap named `app-config` with the file content: `database_host: postgres.production.svc.cluster.local` 2. Mount this ConfigMap as a file at `/etc/app/config.yaml` in the pod 3. Verify the pod starts successfully

The pod manifest already exists; you only need to update it.

Time: 8 minutes

Answer

Create ConfigMap:

```
k create configmap app-config --from-literal=config.yaml='database_host: postgres.production.svc.cluster.local' -n production
```

Or more naturally, create the file first:

```
echo 'database_host: postgres.production.svc.cluster.local' > config.yaml
k create configmap app-config --from-file=config.yaml -n production
```

Update pod manifest to mount ConfigMap. The pod's `volumes` section should reference the ConfigMap, and the container's `volumeMounts` should mount it:

```
spec:
  containers:
    - name: app
      image: your-image
      volumeMounts:
        - name: config-volume
          mountPath: /etc/app
  volumes:
    - name: config-volume
      configMap:
        name: app-config
        items:
          - key: config.yaml
            path: config.yaml
```

Verify:

```
k get pod app-frontend -n production
k exec app-frontend -n production -- cat /etc/app/config.yaml
```

Key insight: The pod was crashing because the file didn't exist. ConfigMap mounts it at the expected path. The `items` field ensures the key name matches the file name.

Q2. RBAC and Service Account

A developer needs to be able to list Deployments and Pods in the `staging` namespace, but not in other namespaces. Currently, they cannot perform these actions.

Create: 1. A Role that allows listing Pods and Deployments in `staging` 2. A RoleBinding that grants this Role to the user `dev-user`

Verify the permissions work using `kubectl auth can-i`.

Time: 7 minutes

Answer

Create Role:

```
k create role list-resources --verb=list --resource=pods,deployments -n staging
```

Create RoleBinding:

```
k create rolebinding dev-user-binding --role=list-resources --user=dev-user -n staging
```

Verify permissions:

```
k auth can-i list pods --as=dev-user -n staging
k auth can-i list deployments --as=dev-user -n staging
k auth can-i list pods --as=dev-user -n production # Should be denied
```

Key insight: RoleBinding is namespace-scoped. `--as=dev-user` tests permissions for that user. The user cannot list pods in other namespaces because the Role only applies to `staging`.

Q3. Node Maintenance

Node `worker-2` needs maintenance and will be offline for 1 hour. Currently, there are several running pods on this node including a critical database pod and a deployment with 3 replicas.

1. Prepare the node for maintenance without losing pod data
2. Ensure the database pod is NOT evicted (it has local persistent data)
3. Verify the node is ready for maintenance

Time: 6 minutes

Answer

Cordon the node:

```
k cordon worker-2
```

Drain the node but exclude static pods (which can't be evicted):

```
k drain worker-2 --ignore-daemonsets --delete-emptydir-data
```

If the database pod has a local persistent volume and shouldn't be evicted, add `--skip-wait-for-delete-timeout`:

```
k drain worker-2 --ignore-daemonsets --delete-emptydir-data --force
```

Verify node is cordoned:

```
k get node worker-2 # Should show NotReady,SchedulingDisabled
```

After maintenance, uncordon:

```
k uncordon worker-2
```

Key insight: `cordon` prevents new pods from scheduling. `drain` evicts existing pods. Daemonsets pods need `--ignore-daemonsets` because they manage themselves. Pods with local storage may need `--force` to evict.

Q4. Persistent Volume Configuration

An application needs persistent storage that: 1. Size: 5Gi 2. Access mode: can be used by multiple pods simultaneously 3. Storage class: `fast-ssd` 4. Mounted at `/data` in the pod

Create the necessary resources (StorageClass if needed, PV, PVC, and update a deployment to use it). The deployment `db-app` already exists in the `databases` namespace.

Time: 10 minutes

Answer

Assuming you need a ReadWriteMany storage class for multiple pods:

Create StorageClass (if `fast-ssd` doesn't exist):

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd
provisioner: pd.csi.storage.gke.io
parameters:
  type: pd-ssd
allowVolumeExpansion: true
```

Create PVC:

```
k create pvc db-storage --size=5Gi --storageclass=fast-ssd --access-modes=ReadWriteMany -n databases
```

Or YAML:

```

apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: db-storage
  namespace: databases
spec:
  accessModes:
    - ReadWriteMany
  storageClassName: fast-ssd
  resources:
    requests:
      storage: 5Gi

```

Update deployment `db-app` to mount the PVC:

```

k set volume deployment/db-app -n databases --add --mount-path=/data --type=persistentVolumeClaim --claim-name=db-storage

```

Or edit and add to spec:

```

volumeMounts:
- name: data-volume
  mountPath: /data
...
volumes:
- name: data-volume
  persistentVolumeClaim:
    claimName: db-storage

```

Verify:

```

k get pvc -n databases
k describe pod -n databases # Should show mapped volume

```

Key insight: `ReadWriteMany` is needed if multiple pods use the same storage. PVC claims the storage from the pool. The deployment mounts it at `/data`.

Q5. Network Policy Troubleshooting

Currently, all pods in namespace `shop` can communicate with each other. You need to implement network isolation: 1. Frontend pods (label: `tier=frontend`) can receive traffic on port 80 only 2. Backend pods (label: `tier=backend`) can receive traffic from frontend pods on port 3000 only 3. Database pods (label: `tier=database`) should not receive any incoming traffic except from backend pods on port 5432

Create the appropriate NetworkPolicies.

Time: 10 minutes

Answer

Frontend policy (allow inbound on port 80):

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: frontend-policy
  namespace: shop
spec:
  podSelector:
    matchLabels:
      tier: frontend
  policyTypes:
    - Ingress
  ingress:
    - protocol: TCP
      ports:
        - port: 80

```

Backend policy (allow inbound from frontend on port 3000):

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: backend-policy
  namespace: shop
spec:
  podSelector:
    matchLabels:
      tier: backend
  policyTypes:
  - Ingress
  ingress:
  - protocol: TCP
    ports:
    - port: 3000
    from:
    - podSelector:
        matchLabels:
          tier: frontend

```

Database policy (allow inbound from backend on port 5432):

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: database-policy
  namespace: shop
spec:
  podSelector:
    matchLabels:
      tier: database
  policyTypes:
  - Ingress
  ingress:
  - protocol: TCP
    ports:
    - port: 5432
    from:
    - podSelector:
        matchLabels:
          tier: backend

```

Apply all three policies:

```
k apply -f frontend-policy.yaml -f backend-policy.yaml -f database-policy.yaml -n shop
```

Key insight: NetworkPolicy uses label selectors to specify both source and destination. The `from` field restricts incoming traffic to only those pods. All three policies work together to enforce the communication flow.

Q6. Helm Chart Deployment

A Helm chart named `monitoring` exists in the `helm-charts` directory locally. It has default values that need to be overridden: - Set `replica_count` to 3 - Set `storage_size` to 50Gi - Enable `persistence`

Deploy the chart to namespace `monitoring`. If the namespace doesn't exist, create it. The deployment should be named `monitoring-stack`.

Time: 8 minutes

Answer

Create namespace:

```
k create namespace monitoring
```

Deploy Helm chart with overrides:

```

helm install monitoring-stack ./helm-charts/monitoring \
  -n monitoring \
  --set replica_count=3 \
  --set storage_size=50Gi \
  --set persistence=true

```

Verify deployment:

```
helm list -n monitoring
k get deployment -n monitoring
```

Key insight: `--set` overrides values in the chart's `values.yaml`. Multiple `--set` flags can be chained. `helm list` shows installed releases by name.

Q7. Container Runtime Configuration

The cluster is having issues with container image pulls. After investigation, you realize the cluster needs to use CRI-dockerd instead of the default containerd runtime. The necessary binaries are already installed.

Configure the kubelet on worker nodes to use CRI-dockerd and verify the configuration works.

Time: 8 minutes

Answer

On each worker node, update kubelet config:

```
ssh worker-node
sudo nano /etc/kubernetes/kubelet.conf
```

Add or update:

```
containerRuntimeEndpoint: unix:///var/run/cri-dockerd.sock
```

Restart kubelet:

```
sudo systemctl restart kubelet
```

Verify the runtime:

```
k get nodes -o wide
k describe node worker-1 # Check Runtime Version should show docker
```

Test with a pod:

```
k run test-pod --image=nginx
k get pod test-pod -o wide
```

Key insight: CRI-dockerd listens on a Unix socket at `/var/run/cri-dockerd.sock`. Kubelet must be told where to find it. After restart, pods should run on CRI-dockerd.

Q8. API Server Audit Logs

The cluster has been experiencing unauthorized API requests. You need to enable and review API server audit logs to identify: 1. Who accessed ConfigMaps in the `default` namespace 2. When the access occurred 3. Whether it was successful

Enable API server audit logging and examine the logs for ConfigMap access patterns.

Time: 10 minutes

Answer

Enable audit logging in kube-apiserver. On the control plane node:

```
sudo nano /etc/kubernetes/manifests/kube-apiserver.yaml
```

Add audit policy file and log output:

```
spec:
  containers:
    - command:
      - kube-apiserver
      - --audit-policy-file=/etc/kubernetes/audit-policy.yaml
      - --audit-log-path=/var/log/kubernetes/audit.log
      - --audit-log-maxage=7
      ...
  volumeMounts:
    - name: audit
      mountPath: /var/log/kubernetes
  volumes:
    - name: audit
      hostPath:
        path: /var/log/kubernetes
```

Create audit policy:

```
sudo nano /etc/kubernetes/audit-policy.yaml
```

Policy to log ConfigMap access:

```
apiVersion: audit.k8s.io/v1
kind: Policy
rules:
- level: RequestResponse
  resources:
  - group: ""
    resources: ["configmaps"]
    namespaces: ["default"]
- level: Metadata
  resources:
  - group: ""
    resources: ["configmaps"]
```

Review logs:

```
sudo tail -f /var/log/kubernetes/audit.log | jq '[] | select(.objectRef.resource=="configmaps")'
```

Key insight: Audit logs capture who did what and when. By filtering for ConfigMaps in the `default` namespace, you can identify unauthorized access patterns.

Q9. HPA and Application Scaling

An application deployment named `web-app` in namespace `production` runs with 2 replicas. Create a Horizontal Pod Autoscaler that maintains an average CPU utilization of 70% across pods, scaling between a minimum of 2 and a maximum of 10 replicas.

Time: 7 minutes

Answer

Create HPA:

```
k autoscale deployment web-app -n production --min=2 --max=10 --cpu-percent=70
```

Or YAML for more control:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: web-app-hpa
  namespace: production
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web-app
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
```

Verify HPA:

```
k get hpa -n production
k describe hpa web-app-hpa -n production
kubectl top pods -n production # confirms metrics-server is working
```

Key insight: HPA uses a single target utilization, not separate scale-up and scale-down thresholds. On each sync it computes `desiredReplicas = ceil(currentReplicas × currentCPU / targetCPU)` and scales in either direction within the `minReplicas / maxReplicas` bounds. A built-in tolerance (default 10%) prevents action on minor fluctuations, and scale-down aggressiveness can be tuned via `spec.behavior.scaleDown` (e.g. `stabilizationWindowSeconds`) rather than any fixed percentage. HPA requires `metrics-server` to be running.

Q10. Ingress Configuration with TLS

A service `api` needs to be exposed via Ingress with: 1. Hostname: `api.example.com` 2. TLS enabled with certificate secret `api-tls-cert` 3. Path-based routing: `/api/*` routes to the service 4. Health check endpoint at `/health` should not require TLS termination
Create the Ingress resource (assume the TLS certificate secret already exists).

Time: 8 minutes

Answer

Create Ingress:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: api-ingress
spec:
  tls:
    - hosts:
        - api.example.com
      secretName: api-tls-cert
  rules:
    - host: api.example.com
      http:
        paths:
          - path: /api/
            pathType: Prefix
            backend:
              service:
                name: api
                port:
                  number: 80
          - path: /health
            pathType: Exact
            backend:
              service:
                name: api
                port:
                  number: 80
```

Apply Ingress:

```
k apply -f ingress.yaml
```

Verify:

```
k get ingress
k describe ingress api-ingress
```

Key insight: TLS is configured at the Ingress level. Both `/api/*` and `/health` routes go to the same service. The health endpoint doesn't need special handling; all traffic is TLS-terminated at the Ingress. The service backend receives unencrypted traffic.

Q11. Cluster Troubleshooting

The `kube-scheduler` component has stopped running. Investigate: 1. Check the status of all control plane components 2. Review kubelet logs on the control plane node 3. Identify why the scheduler pod is not running 4. Restart the scheduler if needed

Document the issue and resolution.

Time: 12 minutes

Answer

Check control plane components:

```
k get pods -n kube-system | grep scheduler
k describe pod -n kube-system kube-scheduler-controlplane
k logs -n kube-system kube-scheduler-controlplane
```

Check kubelet logs on control plane:

```
ssh controlplane
sudo journalctl -u kubelet -n 50
sudo journalctl -u kubelet -p err
```

Common causes: - Scheduler CrashLoopBackOff: usually a configuration issue in `/etc/kubernetes/manifests/kube-scheduler.yaml`
- Static pod not starting: check manifest syntax - Kubelet can't read manifest: permissions issue

Restart kubelet:

```
sudo systemctl restart kubelet
```

Verify scheduler is running:

```
k get pods -n kube-system kube-scheduler-controlplane  
k get pods # Verify new pods are being scheduled
```

Key insight: If `kube-scheduler` static pod crashes, no pods can be scheduled. Check kubelet logs to see why the manifest isn't loading. Static pods are in `/etc/kubernetes/manifests/` on the control plane node.

Q12. StatefulSet with Persistent Storage

Deploy a StatefulSet named `database-cluster` with: 1. 3 replicas 2. Each pod needs a persistent volume (PVC) of 10Gi 3. Pods should have stable DNS names like `database-cluster-0.database-cluster`, etc. 4. Service port 5432 should be exposed via a headless service

Time: 10 minutes

Answer

Create StatefulSet:

```

apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: database-cluster
spec:
  serviceName: database-cluster
  replicas: 3
  selector:
    matchLabels:
      app: database
  template:
    metadata:
      labels:
        app: database
    spec:
      containers:
        - name: postgres
          image: postgres:15
          ports:
            - containerPort: 5432
          volumeMounts:
            - name: db-storage
              mountPath: /var/lib/postgresql/data
      volumeClaimTemplates:
        - metadata:
            name: db-storage
          spec:
            accessModes: ["ReadWriteOnce"]
            resources:
              requests:
                storage: 10Gi

```

Q13. Resource Quotas and Limits

The `test` namespace is consuming too many resources. Implement:

1. ResourceQuota that limits the namespace to 4 CPU cores and 8Gi memory total
2. LimitRange that ensures individual pods request at least 100m CPU and 128Mi memory
3. Verify that a pod requesting 2 CPUs and 2Gi memory is rejected

****Time: 8 minutes****

Answer

Create ResourceQuota:

```

```yaml
apiVersion: v1
kind: ResourceQuota
metadata:
 name: test-quota
 namespace: test
spec:
 hard:
 requests.cpu: "4"
 requests.memory: 8Gi
 limits.cpu: "4"
 limits.memory: 8Gi

```

Create LimitRange:

```

apiVersion: v1
kind: LimitRange
metadata:
 name: test-limits
 namespace: test
spec:
 limits:
 - min:
 cpu: 100m
 memory: 128Mi
 type: Pod

```

Apply both:

```
k apply -f resourcequota.yaml -f limitrange.yaml -n test
```

Test deployment of a pod requesting 2 CPUs and 2Gi memory:

```
k run test-pod --image=nginx --requests='cpu=2,memory=2Gi' -n test
```

This should be rejected:

```
Error from server (Forbidden): pods "test-pod" is forbidden: exceeded quota: test-quota
```

Verify quotas and limits:

```

k describe resourcequota test-quota -n test
k describe limitrange test-limits -n test

```

**Key insight:** ResourceQuota limits the namespace total. LimitRange limits individual pods. Together, they prevent resource exhaustion at namespace and pod levels.

## Q14. Pod Security Standards

Namespace `restricted` needs to enforce security policies. Configure: 1. Pod Security Standards in `enforce` mode: `restricted` 2. Pod Security Standards in `audit` mode: `baseline` 3. Test by deploying a pod with `privileged: true` and verify it's denied

**Time: 8 minutes**

### Answer

Update namespace to enforce PSS:

```

k label namespace restricted \
 pod-security.kubernetes.io/enforce=restricted \
 pod-security.kubernetes.io/audit=baseline \
 pod-security.kubernetes.io/warn=baseline

```

Or via YAML:

```

apiVersion: v1
kind: Namespace
metadata:
 name: restricted
 labels:
 pod-security.kubernetes.io/enforce: restricted
 pod-security.kubernetes.io/audit: baseline
 pod-security.kubernetes.io/warn: baseline

```

Test with a privileged pod:

```
k run priv-pod --image=nginx --privileged -n restricted
```

Expected result:

```

Error from server (Forbidden): pods "priv-pod" is forbidden:
violates PodSecurityPolicy: privileged: true

```

**Key insight:** Enforcement ( `enforce` ) blocks pods that violate the policy. Audit and warn levels log violations but allow them. `restricted` PSS requires no privileged containers, read-only root filesystem, etc.

## Q15. Multi-Container Pod with Sidecar

Create a pod named `app-with-logger` that: 1. Main container: runs `nginx` image 2. Sidecar container: runs `busybox` that tails `nginx` logs from a shared volume 3. Both containers share a volume at `/var/log/nginx` (for main) and `/logs` (for sidecar) 4. The pods should continue running and logging without errors

**Time: 8 minutes**

### Answer

Create pod:

```
apiVersion: v1
kind: Pod
metadata:
 name: app-with-logger
spec:
 containers:
 - name: nginx
 image: nginx:1.28
 volumeMounts:
 - name: log-volume
 mountPath: /var/log/nginx
 - name: logger
 image: busybox:1.37
 command: ["/bin/sh"]
 args: ["-c", "tail -f /logs/access.log"]
 volumeMounts:
 - name: log-volume
 mountPath: /logs
 volumes:
 - name: log-volume
 emptyDir: {}
```

Apply:

```
k apply -f pod-with-sidecar.yaml
```

Verify:

```
k get pod app-with-logger
k logs app-with-logger -c nginx
k logs app-with-logger -c logger
k exec app-with-logger -c logger -- ls -la /logs
```

**Key insight:** Both containers share the same `log-volume` (`emptyDir`). The main container writes logs to `/var/log/nginx`. The sidecar reads them from `/logs` (same mount, different path). `EmptyDir` storage is ephemeral and deleted when the pod terminates.

## Mock Exam 02

### Q1. Service Discovery and DNS

An application in namespace `backend` needs to call a service in namespace `frontend` via DNS. The service name is `web-ui` and it exposes port 3000.

The application's configuration needs the full DNS name to connect. What is the correct DNS name within the cluster that the backend application should use?

Also, verify that pod-to-pod DNS resolution works using `nslookup` or `dig` from a test pod.

**Time: 6 minutes**

#### Answer

The correct DNS name format within a cluster is:

```
<service-name>.<namespace>.svc.cluster.local
```

For this case:

```
web-ui.frontend.svc.cluster.local
```

Or simply:

```
web-ui.frontend
```

Both work from within the cluster. Connect with port 3000:

```
web-ui.frontend:3000
```

Verify DNS resolution with `nslookup`:

```
k run debug-pod --image=busybox --rm -it -- nslookup web-ui.frontend
```

Or:

```
k run debug-pod --image=busybox --rm -it -- nslookup web-ui.frontend.svc.cluster.local
```

Expected output shows the service's Cluster IP address. From a test pod in `backend` namespace:

```
k run test -n backend --image=nginx --rm -it -- wget -qO- http://web-ui.frontend:3000
```

**Key insight:** DNS name includes namespace. Short names work only when querying from the same namespace. Full names work cluster-wide. CoreDNS handles this resolution automatically.

### Q2. Deployment Rolling Update and Rollback

A deployment `payment-service` has been updated with a new image version that introduced a bug. Users are reporting errors. You need to: 1. Check the deployment's rollout history 2. Identify which revision had the issue 3. Rollback to the previous stable version 4. Verify the rollback completed successfully

**Time: 8 minutes**

#### Answer

Check rollout history:

```
k rollout history deployment/payment-service
```

Each revision shows the image versions. Identify the problematic revision and check details:

```
k rollout history deployment/payment-service --revision=3
```

Rollback to the previous revision:

```
k rollout undo deployment/payment-service
```

Or rollback to a specific revision:

```
k rollout undo deployment/payment-service --to-revision=2
```

Verify rollback status:

```
k rollout status deployment/payment-service
k get pods -o wide # New pods should be coming up with old image
k describe deployment payment-service # Check image version
```

Check the new revision history:

```
k rollout history deployment/payment-service
```

**Key insight:** Each update creates a new ReplicaSet. Rollback simply scales down the bad ReplicaSet and scales up the good one. Pods are recreated with old image.

### Q3. Secrets and Environment Variables

A pod needs access to database credentials: - Username: `dbuser` - Password: `secret123` - Database: `appdb`

Create a Secret and mount it in a deployment pod as environment variables (not as files).

Verify that the pod can access these environment variables without exposing them in pod logs.

**Time: 7 minutes**

#### Answer

Create Secret:

```
k create secret generic db-creds \
 --from-literal=username=dbuser \
 --from-literal=password=secret123 \
 --from-literal=database=appdb
```

Or via YAML:

```
apiVersion: v1
kind: Secret
metadata:
 name: db-creds
type: Opaque
stringData:
 username: dbuser
 password: secret123
 database: appdb
```

Update deployment to use Secret as environment variables:

```
k set env deployment/app-deploy --from=secret/db-creds --prefix=DB_
```

Or edit deployment to add:

```
spec:
 template:
 spec:
 containers:
 - name: app
 image: myapp:latest
 env:
 - name: DB_USERNAME
 valueFrom:
 secretKeyRef:
 name: db-creds
 key: username
 - name: DB_PASSWORD
 valueFrom:
 secretKeyRef:
 name: db-creds
 key: password
 - name: DB_DATABASE
 valueFrom:
 secretKeyRef:
 name: db-creds
 key: database
```

Verify the environment variables are available (they won't show in pod describe because they're secret):

```
k exec <pod-name> -- env | grep DB_
```

Check pod logs don't expose secrets:

```
k logs <pod-name>
```

**Key insight:** Using `valueFrom.secretKeyRef` ensures secrets aren't exposed in pod definitions. Environment variables are injected at runtime. Never echo secrets in logs.

## Q4. DaemonSet Configuration

A monitoring agent needs to run on every node in the cluster, including control plane nodes. Currently, some nodes are missing the agent due to taints.

Create or update a DaemonSet to: 1. Run the `datadog/agent:latest` image on all nodes 2. Tolerate the taint `node-role.kubernetes.io/control-plane:NoSchedule` 3. Verify the agent is running on all nodes

**Time: 8 minutes**

### Answer

Create DaemonSet with toleration for control plane:

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
 name: monitoring-agent
spec:
 selector:
 matchLabels:
 app: monitoring
 template:
 metadata:
 labels:
 app: monitoring
 spec:
 tolerations:
 - effect: NoSchedule
 key: node-role.kubernetes.io/control-plane
 containers:
 - name: agent
 image: datadog/agent:latest
 resources:
 requests:
 cpu: 100m
 memory: 64Mi
 limits:
 cpu: 200m
 memory: 256Mi
```

Apply:

```
k apply -f daemonset.yaml
```

Verify on all nodes:

```
k get daemonset monitoring-agent
k get pods -o wide -l app=monitoring
```

Should see pods on control plane and all worker nodes:

```
k get pods -A -l app=monitoring
```

Count should match number of nodes:

```
k get nodes --no-headers | wc -l
```

**Key insight:** DaemonSets respect taints only if toleration is specified. Control plane has `node-role.kubernetes.io/control-plane:NoSchedule` taint by default. Without toleration, the pod won't schedule there.

## Q5. Cluster API Server Debugging

The API server appears to be responding slowly. You need to: 1. Check the API server logs for errors or warnings 2. Verify the API server is listening on port 6443 3. Check if there are too many requests from a specific source 4. Review the audit trail for suspicious activity

Provide the commands you would use to debug this issue.

**Time: 9 minutes**

## Answer

Check API server logs:

```
ssh controlplane
sudo journalctl -u kubelet -n 100 # Kubelet logs
sudo journalctl -u kube-apiserver -n 100 # Direct systemd service
```

Or for static pod (common in kubeadm clusters):

```
k logs -n kube-system kube-apiserver-controlplane
k logs -n kube-system kube-apiserver-controlplane --tail=50
```

Verify API server is listening:

```
netstat -tlnp | grep 6443
sudo ss -tlnp | grep apiserver
curl -k https://localhost:6443
```

Check for audit events if audit logging is enabled:

```
sudo tail -f /var/log/kubernetes/audit.log | jq '[] | select(.responseStatus.code >= 500)'
```

Look for request patterns:

```
sudo tail -f /var/log/kubernetes/audit.log | jq '[] | .user.username' | sort | uniq -c
```

Check load on API server:

```
k top nodes
k top pods -n kube-system | grep apiserver
```

Identify slow requests:

```
sudo tail -f /var/log/kubernetes/audit.log | jq '[] | select(.requestReceivedTimestamp < .responseReceivedTimestamp) | .responseReceivedTimestamp - .requestReceivedTimestamp'
```

**Key insight:** API server logs are usually in journalctl for systemd or as a static pod in `-n kube-system`. High latency often indicates etcd performance issues or excessive requests from a client.

## Q6. ConfigMap Updates and Pod Restarts

A ConfigMap contains application configuration that was just updated. Pods currently running the application still have the old configuration because they mounted the ConfigMap at pod startup.

Implement a solution to ensure pods pick up the new configuration without manually restarting them. Consider using volume reload mechanisms or automation.

**Time: 8 minutes**

## Answer

Problem: ConfigMap is mounted as a volume, but changes don't immediately reflect in running pods.

Solution 1 — Manual pod restart:

```
k rollout restart deployment/my-app
```

This recreates all pods which reload the ConfigMap.

Solution 2 — Automatic reloader (Stakater Reloader): This is a third-party tool but shows the pattern:

```
apiVersion: v1
kind: ConfigMap
metadata:
 name: app-config
data:
 config.yaml: |
 key: value
```

Annotation on deployment:

```
spec:
 template:
 metadata:
 annotations:
 configmap.reloader.stakater.com/match: "app-config"
```

When ConfigMap changes, the reloader updates pod annotations, triggering a rollout.

Solution 3 — Ignore for now (reload on next deployment): Some teams accept that ConfigMap changes require manual redeploy. Document this as a known limitation.

For immediate needs:

```
k patch configmap app-config -p '{"data":{"config":"new-value"}}'
k rollout restart deployment/my-app
```

**Key insight:** Kubernetes doesn't automatically restart pods when ConfigMaps change. Volume mounts may update over time (depends on kubelet settings), but the best practice is explicit restart or a reloader controller.

## Q7. PriorityClass and Pod Eviction

Set up a cluster with different priority levels: 1. Create a PriorityClass for critical workloads with priority 1000 2. Create a PriorityClass for standard workloads with priority 100 3. Deploy two critical pods and one standard pod 4. When the node runs out of memory, verify the standard pod is evicted before critical pods

Briefly explain the preemption policy.

**Time: 9 minutes**

### Answer

Create critical PriorityClass:

```
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
 name: critical
value: 1000
globalDefault: false
description: "Critical priority workloads"
preemptionPolicy: PreemptLowerPriority
```

---

### Q8. Service Types and Exposure

A legacy application needs to be accessible from outside the cluster on a specific port. Internal services use ClusterIP, but this application needs external access without using Ingress.

Create a Service of the appropriate type that:

1. Exposes the pod on an external IP
2. Uses port 8080 on the node and routes to port 3000 on the pod
3. Verify external connectivity

**\*\*Time: 7 minutes\*\***

### Answer

Use NodePort Service:

```
```yaml
apiVersion: v1
kind: Service
metadata:
  name: legacy-app
spec:
  type: NodePort
  selector:
    app: legacy
  ports:
    - protocol: TCP
      port: 3000      # Internal pod port
      targetPort: 3000 # Pod listens on this
      nodePort: 8080  # External node port
```

Apply:

```
k apply -f service.yaml
```

Verify:

```
k get svc legacy-app
```

Output shows:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
legacy-app	NodePort	10.100.100.100	<none>	3000:8080/TCP	5s

Access externally:

```
curl http://<node-ip>:8080
```

Verify from outside cluster:

```
curl http://192.168.1.10:8080 # Where 192.168.1.10 is a node IP
```

Key insight: NodePort opens a port on every node. External traffic comes to node:port, then kube-proxy routes to the pod. Port range is typically 30000-32767 (unless custom), but can specify any port in that range.

Q9. CronJob Scheduling

Set up a backup job that should run: 1. Every day at 2 AM 2. Keep only the last 3 successful job runs 3. Suspend the CronJob (it should not run for the next month) 4. Start it again when needed

Create the CronJob manifest and verify the schedule settings.

Time: 7 minutes

Answer

Create CronJob:

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: backup-job
spec:
  schedule: "0 2 * * *" # Every day at 2 AM
  concurrencyPolicy: Forbid
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 1
  suspend: false
  jobTemplate:
    spec:
      template:
        spec:
          containers:
            - name: backup
              image: backup-tool:latest
              command: ["/bin/sh", "-c", "echo 'Running backup'; sleep 5"]
              restartPolicy: OnFailure
```

Apply:

```
k apply -f cronjob.yaml
```

Verify the schedule:

```
k get cronjob backup-job
k describe cronjob backup-job
```

Suspend the CronJob (won't run for next month):

```
k patch cronjob backup-job -p '{"spec":{"suspend":true}}'
```

Resume when needed:

```
k patch cronjob backup-job -p '{"spec":{"suspend":false}}'
```

Check successful job history:

```
k get jobs -l cronjob-name=backup-job
```

Key insight: CronJob schedule uses cron syntax (minute hour day month dayOfWeek). `0 2 * * *` = 2 AM daily.
`successfulJobsHistoryLimit: 3` keeps only last 3 successful jobs; older ones are deleted automatically.

Q10. Cluster Certificate Rotation

The cluster's certificate-based authentication is expiring soon. You need to: 1. Check the current certificate expiration date 2. Renew the certificates using kubeadm 3. Verify the new certificates are in place 4. Ensure the API server is using the new certificates
Provide the commands needed for certificate rotation.

Time: 10 minutes

Answer

Check current certificate expiration:

```
sudo kubeadm certs check-expiration
```

Output shows each certificate and when it expires.

Renew all certificates:

```
sudo kubeadm certs renew all
```

Check they were renewed:

```
sudo kubeadm certs check-expiration
```

Restart the API server and other control plane components to pick up new certs:

```
sudo systemctl restart kubelet
```

Or if static pods:

```
kubect1 -n kube-system delete pod kube-apiserver-<controlplane>
```

Kubelet will automatically recreate it with the new manifest and certs.

Verify API server is using new certs:

```
openssl s_client -connect localhost:6443 </dev/null | openssl x509 -noout -dates
```

Check the notAfter date matches the renewed date.

Key insight: Kubeadm manages certificate renewal. Certificates are typically valid for 1 year from cluster initialization. Renewal doesn't require cluster restart, but API server must reload (usually happens when the pod is deleted/recreated).

Q11. Pod Disruption Budgets

An application deployment runs 10 replicas and cannot lose more than 2 pods simultaneously during cluster disruptions (maintenance, node failures).

Create a PodDisruptionBudget that enforces this constraint. Test the budget by attempting to drain a node with multiple application pods.

Time: 8 minutes

Answer

Create PodDisruptionBudget:

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: app-pdb
spec:
  minAvailable: 8
  selector:
    matchLabels:
      app: my-app
```

This means: at least 8 pods must stay available during disruption. With 10 replicas, only 2 can be disrupted simultaneously.

Apply:

```
k apply -f pdb.yaml
```

Verify:

```
k get pdb
k describe pdb app-pdb
```

Test by draining a node with multiple app pods:

```
k drain worker-1 --ignore-daemonsets
```

Without the PDB, all app pods on this node would be evicted. With the PDB, Kubernetes respects minAvailable and evicts pods gradually, ensuring at least 8 are always running.

Monitor drain progress:

```
k get pods
k get nodes worker-1
```

Key insight: PDB is a soft constraint — it prevents voluntary disruptions (maintenance, scaling) but not hardware failures. Node failure immediately deletes pods and breaks PDB constraints.

Q12. Custom Resource Definitions (CRD)

Your team has created a custom CRD called `Database` to represent database instances. A developer needs to: 1. Check if the CRD already exists 2. Create an instance of the custom resource 3. Verify the custom resource is stored in etcd 4. Delete the custom resource

Provide the commands to interact with the custom resource.

Time: 7 minutes

Answer

Check if CRD exists:

```
k get crd databases.example.com # Specific CRD name
k get crd | grep database
```

Create custom resource instance:

```
apiVersion: example.com/v1
kind: Database
metadata:
  name: prod-db
spec:
  engine: postgres
  version: "15"
  storage: 100Gi
```

Apply:

```
k apply -f database-instance.yaml
```

Verify in etcd and API server:

```
k get databases
k describe database prod-db
k get database prod-db -o yaml
```

Delete the resource:

```
k delete database prod-db
```

Or by name:

```
k delete database --all
```

Key insight: CRDs extend the Kubernetes API with custom resources. Commands work the same as built-in resources. Data is stored in etcd like any other resource.

Q13. Pod Security Policy Bypass

A pod in the `restricted` namespace requires elevated permissions (to run as root). Currently, Pod Security Standards enforce a restricted policy.

1. Create an exception for this specific pod using `pod-security.kubernetes.io` labels
2. Allow the pod to run with elevated privilege
3. Verify other pods in the namespace still follow the restrictive policy

Time: 9 minutes

Answer

Create an exception for a privileged pod using pod-security labels:

Option 1 — Pod-level exception:

```
apiVersion: v1
kind: Pod
metadata:
  name: privileged-pod
  namespace: restricted
  labels:
    pod-security.kubernetes.io/exempt: 'true'
spec:
  containers:
    - name: app
      image: nginx
      securityContext:
        privileged: true
```

Option 2 — Exemption at pod level (if supported by PSS version):

```
k label pod privileged-pod -n restricted pod-security.kubernetes.io/exempt=true
```

Verify the pod runs with elevated privilege:

```
k get pod privileged-pod -n restricted
k exec privileged-pod -- id
```

Verify other pods still follow restrict policy:

```
k run normal-pod --image=nginx -n restricted
# This pod should fail to start if it tries to be privileged
```

Check PSS labels on namespace:

```
k get namespace restricted --show-labels
```

Key insight: Pod security exceptions are label-based. Only pods with the exemption label bypass the policy. Other pods continue to be enforced.

Q14. Storage Class and Dynamic Provisioning

Configure dynamic volume provisioning: 1. Create a StorageClass with the SSD provisioner 2. Create a PVC that requests 20Gi 3. Verify that PV is automatically created and bound 4. Deploy a pod that uses the dynamically provisioned volume

Time: 8 minutes

Answer

Create StorageClass:

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: ssd-provisioner
provisioner: pd.csi.storage.gke.io # GKE example; varies by cluster
parameters:
  type: pd-ssd
allowVolumeExpansion: true
reclaimPolicy: Delete
```

Apply:

```
k apply -f storageclass.yaml
```

Create PVC:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-ssd-claim
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: ssd-provisioner
  resources:
    requests:
      storage: 20Gi
```

Apply:

```
k apply -f pvc.yaml
```

Verify PV is created automatically:

```
k get pvc my-ssd-claim
k get pv # Should see a new PV bound to the PVC
```

Deploy a pod using the PVC:

```
apiVersion: v1
kind: Pod
metadata:
  name: app-using-ssd
spec:
  containers:
    - name: app
      image: nginx
      volumeMounts:
        - name: ssd-volume
          mountPath: /data
  volumes:
    - name: ssd-volume
      persistentVolumeClaim:
        claimName: my-ssd-claim
```

Apply and verify:

```
k apply -f pod.yaml
k get pods
k exec app-using-ssd -- df -h /data # Should show 20Gi mount
```

Key insight: StorageClass enables dynamic provisioning. When PVC is created, the provisioner automatically creates a matching PV. Pod binds to the PVC, which is bound to the PV.

Q15. Namespace Quota and Resource Management

Set up resource governance for the `batch-jobs` namespace: 1. Limit total CPU to 10 cores 2. Limit total memory to 20Gi 3. Set per-pod minimum: 500m CPU, 256Mi memory 4. Set per-pod maximum: 4 CPU, 8Gi memory 5. Test by trying to deploy a pod that exceeds limits and verify it's rejected

Time: 10 minutes

Answer

Create ResourceQuota:

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: batch-jobs-quota
  namespace: batch-jobs
spec:
  hard:
    requests.cpu: "10"
    requests.memory: 20Gi
    limits.cpu: "10"
    limits.memory: 20Gi
```

Create LimitRange:

```
apiVersion: v1
kind: LimitRange
metadata:
  name: batch-jobs-limits
  namespace: batch-jobs
spec:
  limits:
  - type: Container
    min:
      cpu: 500m
      memory: 256Mi
    max:
      cpu: 4
      memory: 8Gi
```

Apply both:

```
k apply -f quota.yaml -f limitrange.yaml -n batch-jobs
```

Verify:

```
k describe quota -n batch-jobs
k describe limits -n batch-jobs
```

Test by deploying a pod that exceeds limits:

```
k run oversized-pod \
-n batch-jobs \
--image=nginx \
--requests='cpu=8,memory=16Gi' \
--overrides='{ "spec": { "containers": [ { "resources": { "limits": { "cpu": "8", "memory": "16Gi" } } } ] } }'
```

Should be rejected:

```
Error: the container cannot be created because the request exceeds the allocation set by the LimitRange
```

Or:

```
Error: the deployment violates the ResourceQuota: requests.cpu and memory exceeded
```

Try again with acceptable values:

```
k run compliant-pod \
-n batch-jobs \
--image=nginx \
--requests='cpu=2,memory=4Gi'
```

Should succeed.

Key insight: LimitRange applies to individual containers. ResourceQuota applies to the whole namespace. Together they prevent resource exhaustion at both pod and namespace levels.

Practice Exercises

Exercise 01 — Pod Basics

1. Create a namespace called `exercise-01`
2. Create a pod named `web` in namespace `exercise-01` with these specs: - Image: `nginx:1.27` - Labels: `app=web`, `tier=frontend` - Resource requests: 64Mi memory, 100m CPU - Resource limits: 128Mi memory, 250m CPU
3. Verify the pod is running and has the correct labels
4. Get the pod's IP address using `-o wide`
5. Add a new label `version=v1` to the running pod
6. Remove the `tier` label from the pod

Answer

```
# Create namespace
k create ns exercise-01

# Generate pod YAML
k run web -n exercise-01 --image=nginx:1.28 --labels=app=web,tier=frontend $do > pod.yaml
```

Edit `pod.yaml` to add resources:

```
apiVersion: v1
kind: Pod
metadata:
  name: web
  namespace: exercise-01
  labels:
    app: web
    tier: frontend
spec:
  containers:
  - name: web
    image: nginx:1.27
    resources:
      requests:
        memory: "64Mi"
        cpu: "100m"
      limits:
        memory: "128Mi"
        cpu: "250m"
```

```
k apply -f pod.yaml

# Verify
k get pod web -n exercise-01 --show-labels
k get pod web -n exercise-01 -o wide

# Add label
k label pod web -n exercise-01 version=v1

# Remove label
k label pod web -n exercise-01 tier-
```

Exercise 02 — Multi-Container Pod (Sidecar Logging)

1. Create a namespace called `exercise-02`
2. Create a pod named `logger` in namespace `exercise-02` with: - An init container (sidecar) named `log-agent` using `busybox:1.36`
 - Set `restartPolicy: Always` on the init container to make it a native sidecar
 - Command: `tail -F /var/log/app/app.log` (capital F waits for file creation)
 - Mount a shared volume at `/var/log/app`
 - A main container named `app` using `busybox:1.36`
 - Command: write a line to `/var/log/app/app.log` every 3 seconds
 - Mount the same shared volume at `/var/log/app`
 - Use an `emptyDir` volume named `log-volume`
3. Verify both containers are running
4. Check the sidecar's logs to see the streamed output

Answer

```
apiVersion: v1
kind: Pod
metadata:
  name: logger
  namespace: exercise-02
spec:
  initContainers:
    - name: log-agent
      image: busybox:1.37
      restartPolicy: Always
      command: ["sh", "-c", "tail -F /var/log/app/app.log"]
      volumeMounts:
        - name: log-volume
          mountPath: /var/log/app
  containers:
    - name: app
      image: busybox:1.37
      command: ["sh", "-c", "while true; do echo \"$(date) - app running\" >> /var/log/app/app.log; sleep 3; done"]
      volumeMounts:
        - name: log-volume
          mountPath: /var/log/app
  volumes:
    - name: log-volume
      emptyDir: {}
```

```
k create ns exercise-02
k apply -f logger.yaml

# Wait for pod to be ready
k get pod logger -n exercise-02 -w

# Check sidecar logs
k logs logger -n exercise-02 -c log-agent
```

Exercise 03 — ConfigMap and Secret

1. Create a namespace called `exercise-03`
2. Create a ConfigMap named `app-config` with: - Key `APP_MODE` = `production` - Key `LOG_LEVEL` = `debug`
3. Create a Secret named `db-creds` with: - Key `DB_USER` = `admin` - Key `DB_PASS` = `s3cretP@ss`
4. Create a pod named `app` that: - Uses image `busybox:1.36`, command `sleep 3600` - Loads `APP_MODE` and `LOG_LEVEL` from the ConfigMap as env vars - Loads `DB_USER` and `DB_PASS` from the Secret as env vars - Mounts the entire ConfigMap as files at `/etc/config/`
5. Verify the env vars are set inside the pod
6. Verify the mounted files exist at `/etc/config/`

Answer

```
k create ns exercise-03

k create configmap app-config -n exercise-03 \
  --from-literal=APP_MODE=production \
  --from-literal=LOG_LEVEL=debug

k create secret generic db-creds -n exercise-03 \
  --from-literal=DB_USER=admin \
  --from-literal=DB_PASS=s3cretP@ss
```

```

apiVersion: v1
kind: Pod
metadata:
  name: app
  namespace: exercise-03
spec:
  containers:
  - name: app
    image: busybox:1.36
    command: ["sleep", "3600"]
    envFrom:
    - configMapRef:
        name: app-config
    - secretRef:
        name: db-creds
    volumeMounts:
    - name: config-vol
      mountPath: /etc/config
  volumes:
  - name: config-vol
    configMap:
      name: app-config

```

```
k apply -f app.yaml
```

```

# Verify
k exec app -n exercise-03 -- env | grep -E "APP_MODE|LOG_LEVEL|DB_USER|DB_PASS"
k exec app -n exercise-03 -- ls /etc/config/

```

Exercise 04 — RBAC

1. Create a namespace called `exercise-04`
2. Create a ServiceAccount named `dev-sa` in namespace `exercise-04`
3. Create a Role named `pod-manager` in namespace `exercise-04` that allows: - `get, list, watch, create, delete` on `pods` - `get, list` on `services`
4. Create a RoleBinding named `dev-pod-access` that binds `pod-manager` to `dev-sa`
5. Verify that `dev-sa` can list pods in `exercise-04`
6. Verify that `dev-sa` cannot list pods in `default` namespace
7. Create a ClusterRole named `node-viewer` that allows `get, list` on `nodes`
8. Create a ClusterRoleBinding named `dev-node-access` binding `node-viewer` to `dev-sa`
9. Verify that `dev-sa` can now list nodes

Answer

```
k create ns exercise-04

# ServiceAccount
k create sa dev-sa -n exercise-04

# Role - Option 1: Create via YAML to ensure separate rules
cat <<EOF | k apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-manager
  namespace: exercise-04
rules:
- apiGroups:
  - ""
  resources:
  - pods
  verbs:
  - get
  - list
  - watch
  - create
  - delete
- apiGroups:
  - ""
  resources:
  - services
  verbs:
  - get
  - list
EOF

# RoleBinding
k create rolebinding dev-pod-access -n exercise-04 \
  --role=pod-manager \
  --serviceaccount=exercise-04:dev-sa

# Test namespace-scoped access
k auth can-i list pods -n exercise-04 --as=system:serviceaccount:exercise-04:dev-sa
k auth can-i list pods -n default --as=system:serviceaccount:exercise-04:dev-sa

# ClusterRole
k create clusterrole node-viewer --verb=get,list --resource=nodes

# ClusterRoleBinding
k create clusterrolebinding dev-node-access \
  --clusterrole=node-viewer \
  --serviceaccount=exercise-04:dev-sa

# Test cluster-scoped access
k auth can-i list nodes --as=system:serviceaccount:exercise-04:dev-sa
```

Exercise 05 — NetworkPolicy

1. Create a namespace called `exercise-05`
2. Deploy two pods: - `frontend` with image `nginx:1.27` and label `role=frontend` - `backend` with image `nginx:1.27` and label `role=backend`
3. Verify that `frontend` can reach `backend` on port 80 (should work before any policy)
4. Create a NetworkPolicy named `backend-policy` that: - Applies to pods with label `role=backend` - Allows ingress only from pods with label `role=frontend` on port 80 - Allows egress to DNS (UDP port 53) — if you skip this, DNS breaks
5. Verify that `frontend` can still reach `backend`
6. Create a third pod `attacker` with label `role=attacker` and verify it cannot reach `backend`

Answer

```
k create ns exercise-05

# Create pods
k run frontend -n exercise-05 --image=nginx:1.27 --labels=role=frontend
k run backend -n exercise-05 --image=nginx:1.27 --labels=role=backend
k run attacker -n exercise-05 --image=nginx:1.27 --labels=role=attacker

# Wait for all pods
k get pods -n exercise-05 -w
```

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: backend-policy
  namespace: exercise-05
spec:
  podSelector:
    matchLabels:
      role: backend
  policyTypes:
    - Ingress
    - Egress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              role: frontend
      ports:
        - protocol: TCP
          port: 80
  egress:
    - to: []
      ports:
        - protocol: UDP
          port: 53
```

```
k apply -f netpol.yaml

# Test
BACKEND_IP=$(k get pod backend -n exercise-05 -o jsonpath='{.status.podIP}')
k exec frontend -n exercise-05 -- curl -m2 http://$BACKEND_IP
k exec attacker -n exercise-05 -- curl -m2 http://$BACKEND_IP
```

Exercise 06 — Deployment Rolling Update and Rollback

1. Create a namespace called `exercise-06`
2. Create a Deployment named `webapp` with: - 3 replicas - Image: `nginx:1.28` - Strategy: RollingUpdate with `maxSurge=1`, `maxUnavailable=0` - Record the change cause
3. Verify all 3 replicas are running
4. Update the image to `nginx:1.29` and record the change
5. Watch the rollout status
6. Check rollout history — you should see 2 revisions
7. Rollback to the previous version (`nginx:1.28`)
8. Verify the rollback worked by checking the image

Answer

```
k create ns exercise-06

# Create deployment
k create deployment webapp -n exercise-06 --image=nginx:1.28 --replicas=3 $do > deploy.yaml
```

Edit `deploy.yaml` to add rolling update strategy, then apply:

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: webapp
  namespace: exercise-06
spec:
  replicas: 3
  selector:
    matchLabels:
      app: webapp
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app: webapp
    spec:
      containers:
        - name: nginx
          image: nginx:1.28

```

```

k apply -f deploy.yaml
# First annotation – no --overwrite needed because the key doesn't exist yet
k annotate deployment webapp -n exercise-06 kubernetes.io/change-cause="initial deploy nginx:1.28"

# Verify
k get deploy webapp -n exercise-06
k get pods -n exercise-06

# Update image
k set image deployment/webapp nginx=nginx:1.29 -n exercise-06
# Second annotation – MUST use --overwrite because kubernetes.io/change-cause already exists on the Deployment object
# The annotation key lives on the Deployment, not on individual revisions, so it persists across updates
k annotate deployment webapp -n exercise-06 kubernetes.io/change-cause="update to nginx:1.29" --overwrite

# Watch rollout
k rollout status deployment/webapp -n exercise-06

# Check history
k rollout history deployment/webapp -n exercise-06

# Rollback
k rollout undo deployment/webapp -n exercise-06

# Verify rollback
k get deploy webapp -n exercise-06 -o jsonpath='{.spec.template.spec.containers[0].image}'
# Should output: nginx:1.28

```

Exercise 07 — StatefulSet Deployment

1. Create a headless Service for the StatefulSet (required for DNS routing)
2. Create a StatefulSet with 3 replicas using the `nginx:1.28` image
3. Verify pods start in order (mysql-0 first, then mysql-1, then mysql-2)
4. Verify each pod has a persistent volume claim attached
5. Delete a pod and observe it respawn with the same name and storage
6. Scale the StatefulSet up and down using `kubectl scale`
7. Delete the StatefulSet and verify PVCs persist (data survives)

Answer

Create a headless Service:

```

apiVersion: v1
kind: Service
metadata:
  name: mysql
  labels:
    app: mysql
spec:
  ports:
    - port: 3306
      name: mysql
  clusterIP: None # This makes it headless
  selector:
    app: mysql

```

Create the StatefulSet:

```

apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mysql
spec:
  serviceName: mysql
  replicas: 3
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
        - name: nginx
          image: nginx:1.28
          ports:
            - containerPort: 3306
              name: mysql
          volumeMounts:
            - name: data
              mountPath: /data
      volumeClaimTemplates:
        - metadata:
            name: data
          spec:
            accessModes: [ "ReadWriteOnce" ]
            resources:
              requests:
                storage: 1Gi

```

Deploy with imperative commands (if needed):

```

# Apply the YAML files
kubectl apply -f service.yaml
kubectl apply -f statefulset.yaml

# Watch pods start in order
k get pods -w

# Verify pod identities persist
k get pods -l app=mysql -o custom-columns=NAME:.metadata.name,IP:.status.podIP

# Check PVCs exist
k get pvc

# Test DNS
k run -it --image=busybox:1.37 --rm -- nslookup mysql-0.mysql

# Scale up
k scale statefulset mysql --replicas=4

# Scale down
k scale statefulset mysql --replicas=2

```

Exercise 08 — Node Drain and Cordon

1. List all nodes and identify a worker node
2. Cordon the worker node (mark it unschedulable)
3. Verify the node shows `SchedulingDisabled`
4. Create a Deployment with 3 replicas and observe where pods are scheduled
5. Drain the worker node — handle DaemonSets and local data
6. Verify all non-DaemonSet pods have been evicted from the node
7. Uncordon the node
8. Scale the deployment to 6 replicas and verify pods get scheduled on the uncordoned node

Answer

```
# List nodes
k get nodes

# Pick a worker node, e.g., worker-1

# Cordon
k cordon worker-1
k get nodes
# worker-1 should show SchedulingDisabled

# Create deployment
k create deployment drain-test --image=nginx:1.27 --replicas=3

# Check pod placement
k get pods -o wide

# Drain
k drain worker-1 --ignore-daemonsets --delete-emptydir-data

# Verify eviction
k get pods -o wide
# All drain-test pods should be on other nodes

# Check node
k get pods -A --field-selector spec.nodeName=worker-1
# Only DaemonSet pods should remain

# Uncordon
k uncordon worker-1
k get nodes
# worker-1 should be Ready (no SchedulingDisabled)

# Scale up — new pods should land on worker-1 too
k scale deployment drain-test --replicas=6
k get pods -o wide
```

Exercise 09 — kubeadm Cluster Upgrade

1. Check the current cluster version (`k get nodes`, `kubeadm version`)
2. Upgrade the control plane node: a. Update the kubeadm package to the target version b. Run `kubeadm upgrade plan` to see available upgrades c. Run `kubeadm upgrade apply v1.35.x` d. Upgrade kubelet and kubectl packages e. Restart kubelet
3. Upgrade a worker node: a. Drain the worker node b. Update kubeadm, kubelet, kubectl packages on the worker c. Run `kubeadm upgrade node` d. Restart kubelet e. Uncordon the worker
4. Verify all nodes show the new version

Answer

```
# === CONTROL PLANE NODE ===

# Check current version
k get nodes
kubeadm version

# Update kubeadm (Ubuntu/Debian example)
sudo apt-mark unhold kubeadm
sudo apt-get update
sudo apt-get install -y kubeadm=1.35.0-1.1
sudo apt-mark hold kubeadm

# Plan
sudo kubeadm upgrade plan

# Apply
sudo kubeadm upgrade apply v1.35.0

# Update kubelet + kubect1
sudo apt-mark unhold kubelet kubect1
sudo apt-get install -y kubelet=1.35.0-1.1 kubect1=1.35.0-1.1
sudo apt-mark hold kubelet kubect1

# Restart kubelet
sudo systemctl daemon-reload
sudo systemctl restart kubelet

# Verify control plane
k get nodes

# === WORKER NODE (run from control plane, then SSH to worker) ===

# From control plane: drain worker
k drain worker-1 --ignore-daemonsets --delete-emptydir-data

# SSH to worker node
ssh worker-1

# Update packages on worker
sudo apt-mark unhold kubeadm kubelet kubect1
sudo apt-get update
sudo apt-get install -y kubeadm=1.35.0-1.1 kubelet=1.35.0-1.1 kubect1=1.35.0-1.1
sudo apt-mark hold kubeadm kubelet kubect1

# Upgrade node config
sudo kubeadm upgrade node

# Restart kubelet
sudo systemctl daemon-reload
sudo systemctl restart kubelet

# Exit back to control plane
exit

# Uncordon worker
k uncordon worker-1

# Verify
k get nodes
```

Remember: - Control plane: `kubeadm upgrade apply` - Worker: `kubeadm upgrade node` - Always drain workers before upgrading them - Always uncordon after

Exercise 10 — Static Pod

1. Find the static pod manifest directory on the node
2. Create a static pod manifest file for a pod named `static-web` with: - Image: `nginx:1.27` - Port: 80
3. Verify the static pod appears in `k get pods` (it will have the node name appended)
4. Try to delete the static pod with `kubect1` — observe what happens

5. Delete the static pod by removing the manifest file
6. Verify the pod is gone

Answer

```
# Step 1: Find static pod path
cat /var/lib/kubelet/config.yaml | grep staticPodPath
# Usually: staticPodPath: /etc/kubernetes/manifests

# Step 2: Create the manifest
sudo tee /etc/kubernetes/manifests/static-web.yaml <<EOF
apiVersion: v1
kind: Pod
metadata:
  name: static-web
  labels:
    app: static-web
spec:
  containers:
  - name: web
    image: nginx:1.27
    ports:
    - containerPort: 80
EOF

# Step 3: Verify (wait a few seconds for kubelet to pick it up)
k get pods | grep static-web
# Output: static-web-<node-name> 1/1 Running

# Step 4: Try to delete via kubectl
k delete pod static-web-<node-name>
# The pod will reappear – kubelet recreates it

# Step 5: Actually delete it
sudo rm /etc/kubernetes/manifests/static-web.yaml

# Step 6: Verify it's gone
k get pods | grep static-web
# No output – pod is gone
```

Exercise 11 – Troubleshoot Cluster Components

Scenario A: Broken kubelet

1. SSH to a worker node
2. Stop the kubelet service
3. From the control plane, observe the node status change to `NotReady`
4. SSH back to the worker and check kubelet logs
5. Restart kubelet and verify the node comes back to `Ready`

Scenario B: CoreDNS troubleshooting

1. Create a pod and try to resolve a service name — should fail if CoreDNS is broken
2. Check CoreDNS pods in `kube-system`
3. Check CoreDNS logs for errors
4. Verify the CoreDNS ConfigMap for misconfigurations
5. Check that the `kube-dns` service has endpoints
6. Fix any issues and verify DNS resolution works

Scenario C: kube-proxy

1. Check if kube-proxy pods are running on all nodes
2. Check kube-proxy logs
3. Verify kube-proxy ConfigMap
4. Check iptables rules on a node

Scenario D: API Server audit logs (Kubernetes 1.35 troubleshooting)

1. Find where audit logs are stored on the control plane node
2. Check the audit policy configuration in the apiserver manifest
3. Look for failed authentication attempts in the audit log

4. Find requests that were denied by RBAC
5. Correlate user identity to request details

Answer

Scenario A: kubelet

```
# SSH to worker
ssh worker-1

# Stop kubelet
sudo systemctl stop kubelet

# From control plane (different terminal)
k get nodes
# worker-1 shows NotReady after ~40 seconds

# Back on worker – check why
sudo systemctl status kubelet
sudo journalctl -u kubelet --no-pager -1 | tail -30

# Fix: restart kubelet
sudo systemctl start kubelet

# Back on control plane
k get nodes
# worker-1 should be Ready again
```

Scenario B: CoreDNS

```
# Check CoreDNS pods
k get pods -n kube-system -l k8s-app=kube-dns

# Check logs
k logs -n kube-system -l k8s-app=kube-dns

# Check ConfigMap
k get configmap coredns -n kube-system -o yaml

# Check endpoints
k get endpoints kube-dns -n kube-system
# Should show IPs of CoreDNS pods

# Test DNS
k run test-dns --image=busybox:1.37 --rm -it -- nslookup kubernetes.default.svc.cluster.local
```

Scenario C: kube-proxy

```
# Check DaemonSet
k get ds kube-proxy -n kube-system

# Check pods
k get pods -n kube-system -l k8s-app=kube-proxy -o wide

# Check logs
k logs -n kube-system -l k8s-app=kube-proxy | tail -20

# Check ConfigMap
k get configmap kube-proxy -n kube-system -o yaml

# On a node, check iptables
sudo iptables-save | grep -c KUBE
```

Scenario D: API Server audit logs

```
# SSH to control plane node
ssh control-plane

# Check apiserver manifest for audit configuration
sudo cat /etc/kubernetes/manifests/kube-apiserver.yaml | grep -i audit

# Find audit log file (look for --audit-log-path flag)
sudo cat /var/log/audit/audit.log | jq . | head -50

# Find failed authentication attempts
sudo cat /var/log/audit/audit.log | jq 'select(.user.username=="unknown" or .status.code>=400)' | head -20

# Find specific user actions (e.g., resource deletion)
sudo tail -100 /var/log/audit/audit.log | jq 'select(.verb=="delete")'

# Check request count by verb
sudo cat /var/log/audit/audit.log | jq -r '.verb' | sort | uniq -c | sort -rn

# Search for RBAC denials
sudo cat /var/log/audit/audit.log | jq 'select(.stage=="ResponseComplete" and .status.code>=400)'
```

Exercise 12 — Storage: PV, PVC, and StorageClass

1. Create a namespace called `exercise-12`
2. Create a PersistentVolume named `my-pv` with: - Capacity: 1Gi - AccessMode: ReadWriteOnce - StorageClass: `manual` - hostPath: `/data/exercise-12` - Reclaim policy: Retain
3. Create a PersistentVolumeClaim named `my-pvc` in namespace `exercise-12`: - Request: 500Mi - AccessMode: ReadWriteOnce - StorageClass: `manual`
4. Create a pod named `storage-consumer` that mounts `my-pvc` at `/data`
5. Verify: - The PVC is **Bound** to the PV - The Pod is **Running** (not stuck Pending) — the consumer Pod proves the storage is actually usable
6. Write a test file inside the mounted volume from the Pod
7. Delete the pod, create a new pod with the same PVC, and verify the file persists
8. Delete the PVC and check PV status (should be **Released**, not Available, because policy is Retain)

Answer

```
k create ns exercise-12

# Create the host directory (on the node)
sudo mkdir -p /data/exercise-12
```

```
# pv.yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: my-pv
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  storageClassName: manual
  hostPath:
    path: /data/exercise-12
```

```
# pvc.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-pvc
  namespace: exercise-12
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 500Mi
  storageClassName: manual
```

```
# pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: storage-consumer
  namespace: exercise-12
spec:
  containers:
    - name: consumer
      image: busybox:1.35
      command: ['sleep', '3600'] # Keeps pod running
      volumeMounts:
        - name: data
          mountPath: /data
  volumes:
    - name: data
      persistentVolumeClaim:
        claimName: my-pvc
```

```
k apply -f pv.yaml
k apply -f pvc.yaml

# Verify PVC is Bound BEFORE creating the Pod
k get pvc my-pvc -n exercise-12
# STATUS should be Bound

# Create the consumer Pod
k apply -f pod.yaml

# Wait for Pod to be Running (proves PVC is usable)
k get pods -n exercise-12 storage-consumer
# Should show: storage-consumer 1/1 Running

# Write a test file
k exec storage-consumer -n exercise-12 -- touch /data/test.txt
k exec storage-consumer -n exercise-12 -- sh -c 'echo "data persists" > /data/test.txt'

# Delete and recreate pod to verify persistence
k delete pod storage-consumer -n exercise-12
k apply -f pod.yaml

# Verify file persists
k exec storage-consumer -n exercise-12 -- cat /data/test.txt
# Output: data persists

# Test reclaim policy
k delete pod storage-consumer -n exercise-12
k delete pvc my-pvc -n exercise-12
k get pv my-pv
# STATUS should be Released (not Available) because policy is Retain
```

Exercise 13 — Helm Install, Upgrade, and Rollback

1. Add the Bitnami chart repository and update the repo cache
2. Install NGINX using Helm in a namespace called `exercise-13`: Release name: `web` - Set `replicaCount=2`
3. Verify the release is deployed and 2 pods are running

4. Upgrade the release to `replicaCount=3` and record the change
5. Check the release history and confirm there are 2 revisions
6. Roll back to revision 1
7. Verify the replica count is back to 2

Answer

```
# Add repo
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update

# Install
helm install web bitnami/nginx -n exercise-13 --create-namespace --set replicaCount=2

# Verify
helm list -n exercise-13
k get pods -n exercise-13

# Upgrade
helm upgrade web bitnami/nginx -n exercise-13 --set replicaCount=3

# Check history
helm history web -n exercise-13

# Rollback
helm rollback web 1 -n exercise-13

# Verify rollback
k get deploy -n exercise-13 -o jsonpath='{.items[0].spec.replicas}'
helm history web -n exercise-13
```

Exercise 14 — Kustomize: Base and Overlay

1. Create a directory structure: `kustomize-lab/ base/ deployment.yaml service.yaml kustomization.yaml overlays/ staging/ kustomization.yaml`
2. In the base: - A Deployment named `app` with image `nginx:1.27` and 1 replica - A ClusterIP Service named `app-svc` on port 80 - A `kustomization.yaml` listing both resources
3. In the staging overlay: - Set namespace to `exercise-14` - Patch replicas to 3 - Add a common label `env: staging`
4. Create namespace `exercise-14`
5. Apply the staging overlay with `kubectl apply -k`
6. Verify 3 pods are running in namespace `exercise-14` with label `env=staging`

Answer

```
mkdir -p kustomize-lab/base kustomize-lab/overlays/staging
```

base/deployment.yaml:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: app
  template:
    metadata:
      labels:
        app: app
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          ports:
            - containerPort: 80
```

base/service.yaml:

```

apiVersion: v1
kind: Service
metadata:
  name: app-svc
spec:
  selector:
    app: app
  ports:
    - port: 80
      targetPort: 80

```

base/kustomization.yaml:

```

resources:
- deployment.yaml
- service.yaml

```

overlays/staging/kustomization.yaml:

```

resources:
- ../../base

namespace: exercise-14

commonLabels:
  env: staging

replicas:
- name: app
  count: 3

```

```

k create ns exercise-14
k apply -k kustomize-lab/overlays/staging/

# Verify
k get pods -n exercise-14
k get deploy app -n exercise-14 --show-labels
k get svc app-svc -n exercise-14

```

Exercise 15 — Gateway API: Gateway and HTTPRoute

1. Create a namespace called `exercise-15`
2. Verify a GatewayClass exists in the cluster (depends on your CNI/controller)
3. Create a Gateway named `web-gateway` in namespace `exercise-15`: - Reference the existing GatewayClass - Listen on port 80, protocol HTTP
4. Create a Deployment named `web` with image `nginx:1.28` and 2 replicas
5. Expose it with a ClusterIP Service named `web-svc` on port 80
6. Create an HTTPRoute named `web-route` that: - References `web-gateway` as the parent - Matches path prefix `/` - Routes to `web-svc` on port 80
7. Verify the Gateway status shows `Programmed: True`
8. Verify the HTTPRoute is attached to the Gateway

Answer

```

k create ns exercise-15

# Check GatewayClass
k get gatewayclass

# Note the name, e.g., "eg" for Envoy Gateway or "cilium" for Cilium

```

```
# gateway.yaml
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: web-gateway
  namespace: exercise-15
spec:
  gatewayClassName: eg    # replace with your GatewayClass name
  listeners:
  - name: http
    port: 80
    protocol: HTTP
    allowedRoutes:
      namespaces:
        from: Same
```

```
# Deploy backend
k create deployment web -n exercise-15 --image=nginx:1.28 --replicas=2
k expose deployment web -n exercise-15 --port=80 --target-port=80 --name=web-svc
```

```
# httproute.yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: web-route
  namespace: exercise-15
spec:
  parentRefs:
  - name: web-gateway
    namespace: exercise-15
  rules:
  - matches:
    - path:
        type: PathPrefix
        value: /
    backendRefs:
    - name: web-svc
      port: 80
```

```
k apply -f gateway.yaml
k apply -f httproute.yaml

# Check Gateway status
k get gateway web-gateway -n exercise-15
k get gateway web-gateway -n exercise-15 -o yaml | grep -A5 "conditions"

# Check HTTPRoute
k get httproute web-route -n exercise-15 -o yaml | grep -A5 "parents"
```

Exercise 16 — Horizontal Pod Autoscaler

1. Create a namespace called `exercise-16`
2. Verify the metrics-server is running in the cluster
3. Create a Deployment named `load-app` with: - Image: `registry.k8s.io/hpa-example` (or `nginx:1.27` with resource requests) - 1 replica - CPU request: 50m - CPU limit: 100m
4. Expose it with a ClusterIP Service named `load-svc` on port 80
5. Create an HPA targeting `load-app`: - Min replicas: 1 - Max replicas: 5 - **Target CPU utilization: 50%** (must be specified as `type: Utilization` with `averageUtilization: 50`)
6. Verify the HPA YAML has correct structure (target type is `Utilization`, not `AverageValue`)
7. Generate load against the service and observe the HPA scaling up
8. Stop the load and observe the HPA scaling back down

Answer

```
k create ns exercise-16

# Check metrics-server
k get pods -n kube-system | grep metrics-server
k top nodes    # should return data
```

```
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: load-app
  namespace: exercise-16
spec:
  replicas: 1
  selector:
    matchLabels:
      app: load-app
  template:
    metadata:
      labels:
        app: load-app
    spec:
      containers:
      - name: app
        image: nginx:1.27
        resources:
          requests:
            cpu: "50m"
          limits:
            cpu: "100m"
        ports:
        - containerPort: 80
```

```

k apply -f deployment.yaml

# Create service
k expose deployment load-app -n exercise-16 --port=80 --target-port=80 --name=load-svc

# Option 1: Create HPA imperatively
k autoscale deployment load-app -n exercise-16 --cpu-percent=50 --min=1 --max=5

# Option 2: Create HPA declaratively (to verify structure)
k apply -f - <<EOF
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: load-app
  namespace: exercise-16
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: load-app
  minReplicas: 1
  maxReplicas: 5
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 50
EOF

# Verify HPA structure
k get hpa -n exercise-16 -o yaml
# Check: metrics[0].resource.target.type should be "Utilization"
# Check: metrics[0].resource.target.averageUtilization should be 50

# Generate load (run in a separate terminal)
k run load-gen -n exercise-16 --image=busybox:1.36 --rm -it -- sh -c \
  "while true; do wget -q -O- http://load-svc.exercise-16; done"

# Watch scaling in another terminal
k get hpa -n exercise-16 -w
k get pods -n exercise-16 -w

# Stop load (Ctrl+C on the load-gen terminal)
# Wait a few minutes for scale-down

```

Exercise 17 — kubectl debug: Pod and Node

Part A: Debug a pod

1. Create a namespace called `exercise-17`
2. Create a pod named `broken-app` with image `nginx:1.27`
3. Attach a debug container to the running pod using `kubectl debug: - Image: busybox:1.36 - Target the broken-app container to share process namespace`
4. From inside the debug container, list the processes (you should see the `nginx` process)
5. Check the filesystem and network from the debug container
6. Exit the debug session

Part B: Debug a node

1. Run `kubectl debug` against a node to get a shell with host filesystem access
2. Use `chroot /host` to access the real node root
3. Check kubelet status from the debug session
4. Exit the debug session

Answer

Part A

```
k create ns exercise-17

k run broken-app -n exercise-17 --image=nginx:1.27

# Wait for running
k get pod broken-app -n exercise-17 -w

# Debug with shared process namespace
k debug broken-app -n exercise-17 -it --image=busybox:1.36 --target=broken-app

# Inside the debug container:
ps aux                # should show nginx master + worker processes
ls /proc/1/root/etc/nginx/ # access the target container filesystem
wget -qO- localhost:80   # test connectivity from inside
exit
```

Part B

```
# Get a node name
k get nodes
# e.g., node-1

k debug node/node-1 -it --image=busybox:1.36

# Inside:
chroot /host

# Now you have real node access
systemctl status kubelet
journalctl -u kubelet --no-pager | tail -20
crictl ps
cat /etc/kubernetes/manifests/kube-apiserver.yaml | head -20

exit # exit chroot
exit # exit debug pod
```

Exercise 18 – CRI-dockerd Installation & Configuration

1. Load the required kernel modules: `overlay` and `br_netfilter`
2. Configure kernel networking parameters via `sysctl`
3. Install Docker (required dependency for CRI-dockerd)
4. Download and install CRI-dockerd binary from GitHub releases
5. Create systemd service files for `cri-docker.service` and `cri-docker.socket`
6. Enable and start the cri-docker services
7. Verify the CRI-dockerd socket is listening at `/run/cri-dockerd.sock`

Answer

```

# Step 1: Load kernel modules
sudo modprobe overlay
sudo modprobe br_netfilter

# Verify modules are loaded
lsmod | grep overlay
lsmod | grep br_netfilter

# Step 2: Configure sysctl parameters
sudo tee /etc/sysctl.d/99-kubernetes-cri.conf > /dev/null <<EOF
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
net.ipv6.conf.all.forwarding = 1
EOF

# Apply the changes
sudo sysctl --system

# Verify parameters are applied
sysctl net.bridge.bridge-nf-call-iptables
sysctl net.ipv4.ip_forward
sysctl net.ipv6.conf.all.forwarding

# Step 3: Install Docker (prerequisite for CRI-dockerd)
sudo apt-get update
sudo apt-get install -y docker.io

# Start and enable Docker
sudo systemctl start docker
sudo systemctl enable docker

# Step 4: Download and install CRI-dockerd
VER=$(curl -s https://api.github.com/repos/Mirantis/cri-dockerd/releases/latest | grep tag_name | cut -d '"' -f 4 | sed 's/v//')
echo "Installing CRI-dockerd version: $VER"

wget https://github.com/Mirantis/cri-dockerd/releases/download/v${VER}/cri-dockerd-${VER}.amd64.tgz
tar xzf cri-dockerd-${VER}.amd64.tgz
sudo mv cri-dockerd/cri-dockerd /usr/local/bin/
cri-dockerd --version

# Step 5: Create systemd service file
sudo tee /etc/systemd/system/cri-docker.service > /dev/null <<'EOF'
[Unit]
Description=CRI Docker daemon
After=docker.service network-online.target firewalld.service
Wants=network-online.target
Documentation=https://github.com/Mirantis/cri-dockerd

[Service]
Type=notify
EnvironmentFile=/etc/sysconfig/cri-docker
ExecStart=/usr/local/bin/cri-dockerd --network-plugin=cni
ExecReload=/bin/kill -s HUP $MAINPID
TimeoutStopSec=120
RestartSec=5
Restart=always

[Install]
WantedBy=multi-user.target
EOF

# Step 6: Create systemd socket file
sudo tee /etc/systemd/system/cri-docker.socket > /dev/null <<'EOF'
[Unit]
Description=CRI Docker socket
Documentation=https://github.com/Mirantis/cri-dockerd
PartOf=cri-docker.service

[Socket]
ListenStream=%t/cri-dockerd.sock
SocketMode=0660
SocketUser=root

```

```
SocketGroup=docker
```

```
[Install]
```

```
WantedBy=sockets.target
```

```
EOF
```

```
# Step 7: Enable and start CRI-dockerd services
```

```
sudo systemctl daemon-reload
```

```
sudo systemctl enable cri-docker.service cri-docker.socket
```

```
sudo systemctl start cri-docker.socket
```

```
sudo systemctl start cri-docker.service
```

```
# Step 8: Verify services are running
```

```
sudo systemctl status cri-docker.service
```

```
sudo systemctl status cri-docker.socket
```

```
# Step 9: Verify socket is accessible
```

```
ls -la /run/cri-dockerd.sock
```

```
sudo crictl version
```

```
# Step 10: For kubeadm, use this when initializing or joining:
```

```
# sudo kubeadm init --cri-socket unix:///run/cri-dockerd.sock
```

```
# Or for joining a node:
```

```
# sudo kubeadm join <control-plane-ip>:6443 --token <token> --discovery-token-ca-cert-hash sha256:<hash> --cri-socket  
unix:///run/cri-dockerd.sock
```

Exercise 19 – Classic Ingress

1. Create a namespace called `exercise-19`
2. Create two Deployments: - `web-app` with image `nginx:1.28` and 2 replicas, serving on port 80 - `api-app` with image `nginx:1.28` and 2 replicas, serving on port 8080
3. Expose `web-app` Deployment with a ClusterIP service named `web-service` on port 80
4. Expose `api-app` Deployment with a ClusterIP service named `api-service` on port 8080
5. Create an Ingress resource named `app-ingress` that routes: - `/` traffic to `web-service:80` - `/api` traffic to `api-service:8080` - Default backend (no matching path) should point to `web-service`
6. Verify the Ingress resource was created and has an address assigned
7. Test path-based routing by describing the Ingress (check rules)

Answer

```
# Namespace
k create ns exercise-19

# Deployments
k create deployment web-app -n exercise-19 --image=nginx:1.28 --replicas=2 $do | k apply -f -
k create deployment api-app -n exercise-19 --image=nginx:1.28 --replicas=2 $do | k apply -f -

# Wait for pods
k wait --for=condition=ready pod -l app=web-app -n exercise-19 --timeout=30s

# Services
k expose deployment web-app -n exercise-19 --name=web-service --type=ClusterIP --port=80 --target-port=80
k expose deployment api-app -n exercise-19 --name=api-service --type=ClusterIP --port=8080 --target-port=8080

# Verify endpoints exist
k get endpoints -n exercise-19

# Ingress YAML definition
cat <<EOF | k apply -f -
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
  namespace: exercise-19
spec:
  ingressClassName: nginx
  defaultBackend:
    service:
      name: web-service
      port:
        number: 80
  rules:
  - http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: web-service
            port:
              number: 80
      - path: /api
        pathType: Prefix
        backend:
          service:
            name: api-service
            port:
              number: 8080
EOF

# Verify Ingress was created
k get ingress -n exercise-19
k describe ingress app-ingress -n exercise-19

# Note: ADDRESS will remain <pending> if no ingress controller is running
# In a real cluster with nginx-ingress installed, it would show the external IP/hostname
```

Exercise 20 — Pod Security Standards

1. Create a namespace called `exercise-20`
2. Label it with Pod Security Standards at `restricted` level in `enforce` mode
3. Attempt to run a pod with a privileged container in `exercise-20`—it should be rejected
4. Create a second namespace `exercise-20-baseline` with `baseline` PSS level
5. Run a pod with `runAsNonRoot: true` in the baseline namespace—it should succeed
6. Run a pod with `privileged: true` in the baseline namespace—it should be rejected
7. Create a third namespace `exercise-20-audit` with `restricted` level but in `audit` mode
8. Run a privileged pod in audit mode—it should succeed but be logged as a violation
9. Check the audit annotations on the pod that ran in audit mode

Answer

```
# Create and label namespaces with PSS
k create ns exercise-20
k label ns exercise-20 pod-security.kubernetes.io/enforce=restricted

k create ns exercise-20-baseline
k label ns exercise-20-baseline pod-security.kubernetes.io/enforce=baseline

k create ns exercise-20-audit
k label ns exercise-20-audit \
  pod-security.kubernetes.io/enforce=restricted \
  pod-security.kubernetes.io/audit=restricted \
  pod-security.kubernetes.io/warn=restricted

# Test restricted enforcement
# This command will fail:
kubectl run restricted-test --image=nginx:1.28 -n exercise-20 --dry-run=server
# Error: pods "restricted-test" is forbidden: violates PodSecurityPolicy: ...

# Create compliant pod for restricted namespace
cat <<EOF | k apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: secure-pod
  namespace: exercise-20
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 1000
    fsGroup: 2000
  containers:
  - name: nginx
    image: nginx:1.28
    securityContext:
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities:
        drop:
        - ALL
    volumeMounts:
    - name: tmp
      mountPath: /tmp
    - name: cache
      mountPath: /var/cache/nginx
  volumes:
  - name: tmp
    emptyDir: {}
  - name: cache
    emptyDir: {}
EOF

# Test baseline
kubectl run baseline-pod --image=nginx:1.28 -n exercise-20-baseline
k get pods -n exercise-20-baseline

# Test audit mode
kubectl run audit-test --image=nginx:1.28 -n exercise-20-audit
k describe pod audit-test -n exercise-20-audit
# Look for pod-security.kubernetes.io/restricted annotation showing violations

# Verify labels were applied correctly
k get ns exercise-20 exercise-20-baseline exercise-20-audit -o json | jq '.items[].metadata.labels'
```

Exercise 21 — Jobs and CronJobs

1. Create a Job named `cleanup-job` that runs the image `busybox:1.37` with command `echo "Cleanup complete"`. Set it to run 3 times (`completions=3`) serially (`parallelism=1`).
2. Verify the Job runs to completion (status should show 3/3). Check the pod logs.

3. Create a CronJob named `hourly-backup` that runs `backup-script.sh` every hour. Use the same busybox image. Let it keep 3 successful job histories (`successfulJobsHistoryLimit=3`).
4. List CronJobs and check the next scheduled time.
5. Manually trigger the CronJob once (create a Job from it).
6. Verify the manual Job ran and logged output.

Answer

Step 1: Create the Job

```
k create job cleanup-job \
  --image=busybox:1.37 \
  --command -- echo "Cleanup complete"

# This creates a Job that runs once. For 3 completions:
k patch job cleanup-job --type json -p '[
  {"op": "replace", "path": "/spec/completions", "value": 3},
  {"op": "replace", "path": "/spec/parallelism", "value": 1}
]'
```

Or using a declarative YAML:

```
apiVersion: batch/v1
kind: Job
metadata:
  name: cleanup-job
spec:
  completions: 3
  parallelism: 1
  backoffLimit: 3
  template:
    spec:
      containers:
      - name: cleanup
        image: busybox:1.37
        command: ["echo", "Cleanup complete"]
      restartPolicy: Never
```

Step 2: Verify Job runs

```
k get job cleanup-job -w
# Watch until Completions shows 3/3

k get pod -l job-name=cleanup-job
# Should show 3 pods

k logs job/cleanup-job --all-containers=true
# Shows all pod logs from the Job
```

Step 3: Create CronJob

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: hourly-backup
spec:
  schedule: "0 * * * *" # Every hour at :00
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 1
  jobTemplate:
    spec:
      template:
        spec:
          containers:
          - name: backup
            image: busybox:1.37
            command: ["echo", "Backup at $(date)"]
          restartPolicy: OnFailure
```

```
k apply -f cronjob.yaml
```

Step 4: List CronJobs

```
k get cronjob hourly-backup
# SCHEDULE    SUSPEND    ACTIVE    LAST SCHEDULE    AGE
# 0 * * * *    False      0         <none>           20s
```

Step 5: Manually trigger

```
k create job hourly-backup-manual-1 --from=cronjob/hourly-backup
```

Step 6: Verify manual Job

```
k get job -l cronjob-name=hourly-backup
k logs job/hourly-backup-manual-1
```

Exercise 22 – PriorityClass

1. Create a PriorityClass named `high-priority` with value 1000 and `preemptionPolicy` set to `PreemptLowerPriority`.
2. Create a PriorityClass named `low-priority` with value 10 and `preemptionPolicy` set to `Never`.
3. Create a Pod named `critical-app` using `nginx:1.28` and assign it to the `high-priority` class.
4. Create a Pod named `background-job` using `busybox:1.37` to run `sleep 3600` and assign it to the `low-priority` class.
5. Verify both pods are running. Check their priority values with `kubectl get pod -o json` and filter for `priority` and `priorityClassName`.
6. If possible, drain a node to trigger preemption and observe which pod gets evicted.

Answer

Step 1: Create PriorityClasses

```
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: high-priority
value: 1000
preemptionPolicy: PreemptLowerPriority
globalDefault: false
description: "High priority for critical applications"
---
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: low-priority
value: 10
preemptionPolicy: Never
globalDefault: false
description: "Low priority for background jobs"
```

```
k apply -f priorityclasses.yaml
```

Verify:

```
k get priorityclass
# NAME          VALUE    GLOBAL-DEFAULT    AGE
# high-priority  1000     false             5s
# low-priority   10       false             5s
```

Step 2: Create Pods with PriorityClasses

```

apiVersion: v1
kind: Pod
metadata:
  name: critical-app
spec:
  priorityClassName: high-priority
  containers:
  - name: app
    image: nginx:1.28
---
apiVersion: v1
kind: Pod
metadata:
  name: background-job
spec:
  priorityClassName: low-priority
  containers:
  - name: worker
    image: busybox:1.37
    command: ["sleep", "3600"]

```

```
k apply -f pods.yaml
```

Step 3: Verify Priority

```

k get pod -o wide
k get pod critical-app -o json | jq '.spec.priorityClassName'
# "high-priority"

k get pod critical-app -o json | jq '.spec.priority'
# 1000

k get pod background-job -o json | jq '.spec.priority'
# 10

```

Step 4: Trigger Preemption (optional)

If you have a multi-node cluster, drain a node:

```
k drain <node> --ignore-daemonsets --delete-emptydir-data
```

Monitor with:

```
k get events --sort-by='.lastTimestamp'
```

The `background-job` pod should be evicted (assuming resources force it), and `critical-app` should remain running because it has higher priority.

Step 5: Cleanup

```

k delete pod critical-app background-job
k delete priorityclass high-priority low-priority

```

Exercise 23 — Resource Requests Tuning (Pods Won't Schedule)

1. Create a namespace called `exercise-23`
2. Create a Deployment with: - 3 replicas - Image: `nginx:1.27` - Resource requests: 512Mi memory, 500m CPU (intentionally too high) - Cluster has 2 nodes with limited resources
3. Observe pods pending — they won't schedule
4. Check node capacity and available resources
5. Calculate correct resource requests to fit 3 replicas
6. Update Deployment with new (lower) requests
7. Verify all 3 pods schedule and run
8. Document the calculation

Answer

```
k create ns exercise-23
kn exercise-23

# Create deployment with too-high requests (will fail to schedule)
k create deployment web --image=nginx:1.27 --replicas=3 $do > deploy.yaml
```

Edit `deploy.yaml` to add initial (intentionally high) requests:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
  namespace: exercise-23
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          resources:
            requests:
              memory: "512Mi"
              cpu: "500m"
```

```
k apply -f deploy.yaml

# Check pods – they are Pending
k get pods

# Describe to see error
k describe pod <pod-name>
# Output: "Insufficient memory"

# Check node capacity
k describe node <node-name>
# Allocatable: 2000m (CPU)
#               2Gi (Memory)
# With 3 replicas, need to divide available resources

# Calculate: If node has 2Gi available and 2 cores (2000m)
# Per pod: ~600Mi memory and 600m CPU (with overhead)
# But if already running system pods, reduce further

# Edit deploy.yaml with lower values
```

Fix the requests to something realistic:

```
resources:
  requests:
    memory: "128Mi"
    cpu: "100m"
```

```
k apply -f deploy.yaml

# Verify all 3 pods now Running
k get pods -n exercise-23

# Check actual usage (will be much lower than requests)
k top pods -n exercise-23

# Cleanup
k delete ns exercise-23
```

Exercise 24 — PriorityClass and Patch Operations

1. Create a namespace called `exercise-24`
2. Create a PriorityClass with: - Name: `high-priority` - Value: `999999` (one less than 1000000 — max priority) - Description: "Critical workload"
3. Create a Deployment named `critical-app` : - 1 replica initially - Image: `busybox:1.36` - Do NOT specify `priorityClassName` yet
4. Use `kubectl patch` to add `priorityClassName: high-priority` to the Deployment
5. Verify the pod has the priority class assigned
6. Create a second lower-priority Deployment to compare
7. Verify scheduling order respects priority

Answer

```
k create ns exercise-24
kn exercise-24

# Create PriorityClass
k create priorityclass high-priority --value=999999 --description="Critical workload"

# Verify it was created
k get priorityclass high-priority

# Create deployment WITHOUT priorityClassName
k create deployment critical-app --image=busybox:1.36 --replicas=1 $do > deploy.yaml
# (image is sleep 3600 in original, but busybox defaults are fine)

k apply -f deploy.yaml

# Now patch the deployment to add priorityClassName
# Target: spec.template.spec.priorityClassName
k patch deployment critical-app -p '{"spec":{"template":{"spec":{"priorityClassName":"high-priority"}}}}'

# Verify patch was applied
k get deploy critical-app -o jsonpath='{.spec.template.spec.priorityClassName}'
# Output: high-priority

# The pod will be recreated with the new priority class
k get pods -n exercise-24

# Verify pod has it
k get pod -n exercise-24 -o jsonpath='{.items[0].spec.priorityClassName}'
# Output: high-priority

# Check priority value
k get priorityclass high-priority -o jsonpath='{.value}'
# Output: 999999

# Cleanup
k delete ns exercise-24
k delete priorityclass high-priority
```

Alternative patch with explicit field path (if JSON patch fails):

```
k patch deployment critical-app --type='json' -p='[{"op": "add", "path": "/spec/template/spec/priorityClassName", "value": "high-priority"}]'
```

Exercise 25 — Storage with WaitForFirstConsumer Binding

1. Create a namespace called `exercise-25`
2. Create a StorageClass named `local-storage` with: - Provisioner: `kubernetes.io/no-provisioner` (local volumes) - Binding mode: `WaitForFirstConsumer`
3. Create a PersistentVolume (PV) with: - Name: `local-pv-1` - Size: `1Gi` - AccessMode: `ReadWriteOnce` - StorageClassName: `local-storage` - Local path: `/tmp/k8s-pv` (or any valid node path)
4. Create a PersistentVolumeClaim (PVC) with: - Name: `local-pvc` - Size: `1Gi` - StorageClassName: `local-storage`
5. Observe: PVC remains Pending (not Bound) — WHY?
6. Create a Pod that uses the PVC: - Image: `busybox:1.36` - Mount PVC at `/data`
7. Observe: Now PVC transitions to Bound - Pod schedules on the node where the PV exists
8. Verify data persistence by writing to the volume

Answer

```

k create ns exercise-25
kn exercise-25

# Create StorageClass with WaitForFirstConsumer
cat <<EOF | k apply -f -
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: local-storage
provisioner: kubernetes.io/no-provisioner
volumeBindingMode: WaitForFirstConsumer
EOF

# Create PersistentVolume
cat <<EOF | k apply -f -
apiVersion: v1
kind: PersistentVolume
metadata:
  name: local-pv-1
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
  storageClassName: local-storage
  local:
    path: /tmp/k8s-pv
  nodeAffinity:
    required:
      nodeSelectorTerms:
        - matchExpressions:
            - key: kubernetes.io/hostname
              operator: In
              values:
                - <node-name> # Replace with actual node name
EOF

# Get node name if needed
k get nodes -o jsonpath='{.items[0].metadata.name}'

# Verify PV created
k get pv

# Create PVC - will be Pending
cat <<EOF | k apply -f -
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: local-pvc
  namespace: exercise-25
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: local-storage
  resources:
    requests:
      storage: 1Gi
EOF

# Check status - should be Pending
k get pvc

# Create Pod using PVC
cat <<EOF | k apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: app-pod
  namespace: exercise-25
spec:
  containers:
    - name: app
      image: busybox:1.36

```

```

    command: ["sleep", "3600"]
    volumeMounts:
    - name: data
      mountPath: /data
  volumes:
  - name: data
    persistentVolumeClaim:
      claimName: local-pvc
EOF

# Now PVC should be Bound
k get pvc -w
# Watch it change from Pending to Bound

# Verify pod is running
k get pod

# Test write
k exec app-pod -n exercise-25 -- sh -c 'echo "hello" > /data/test.txt'
k exec app-pod -n exercise-25 -- cat /data/test.txt

# Cleanup
k delete ns exercise-25
k delete pv local-pv-1
k delete storageclass local-storage

```

Exercise 26 – CRI-dockerd Installation and Configuration

1. SSH into a worker node
2. Install cri-dockerd using the `.deb` package: - Download from GitHub releases - Install the package
3. Configure system for cri-dockerd: - Enable IP forwarding using `sysctl` - Add required kernel modules (`overlay`, `br_netfilter`)
4. Enable and start cri-dockerd service: - `systemctl enable cri-dockerd` - `systemctl start cri-dockerd`
5. Verify cri-dockerd socket is available
6. Configure kubelet to use cri-dockerd
7. Restart kubelet and verify node is Ready

Answer

```
# SSH into worker node
ssh <node-ip>

# 1. Download cri-dockerd (example version)
wget https://github.com/Mirantis/cri-dockerd/releases/download/v0.3.14/cri-dockerd_0.3.14.3-0~ubuntu-focal_amd64.deb

# 2. Install
sudo dpkg -i cri-dockerd_0.3.14.3-0~ubuntu-focal_amd64.deb

# 3. Load required kernel modules
sudo modprobe overlay
sudo modprobe br_netfilter

# 4. Enable IP forwarding
sudo sysctl -w net.ipv4.ip_forward=1
# Persist across reboots
sudo sed -i 's/#net.ipv4.ip_forward=1/net.ipv4.ip_forward=1/' /etc/sysctl.conf
sudo sysctl -p

# 5. Enable and start cri-dockerd
sudo systemctl enable cri-dockerd
sudo systemctl start cri-dockerd

# 6. Verify socket exists
sudo ls -la /run/cri-dockerd.sock

# 7. Check cri-dockerd is running
sudo systemctl status cri-dockerd

# 8. Update kubelet to use cri-dockerd
# Edit /etc/kubernetes/kubelet.conf or /etc/sysconfig/kubelet
sudo sed -i 's|--container-runtime=containerd|--container-runtime=remote|' /etc/kubernetes/kubelet.conf
sudo sed -i '/--container-runtime=remote/a --container-runtime-endpoint=unix:///run/cri-dockerd.sock' /etc/kubernetes/kubelet.conf

# 9. Restart kubelet (full restart, not reload)
sudo systemctl daemon-reload
sudo systemctl restart kubelet

# 10. Check node status (may take 30 seconds)
k get nodes
# Should show Ready status
```

Reference: <https://kubernetes.io/docs/setup/production-environment/container-runtimes/>

Exercise 27 — CNI Installation (Calico/Tigera Operator)

1. Verify cluster CIDR from kube-controller-manager
2. Install Tigera Operator: - Apply operator manifest - Wait for operator deployment
3. Create custom resource for Calico configuration: - Set cluster CIDR matching kube-controller-manager - Configure networking mode (VXLAN or BGP)
4. Apply Calico installation manifest
5. Wait for calico-system pods to be Ready
6. Verify node networking is operational: - Nodes can ping each other - Pods on different nodes can communicate
7. Test NetworkPolicy blocking works

Answer

```
# 1. Get cluster CIDR
k describe cm kubeadm-config -n kube-system | grep -i cidr
# Output: pod-network-cidr=10.244.0.0/16

# 2. Download Tigera operator
curl https://raw.githubusercontent.com/projectcalico/calico/v3.28.0/manifests/tigera-operator.yaml -o tigera-operator.yaml

# 3. Install operator
k apply -f tigera-operator.yaml

# 4. Wait for operator to be ready
k wait --for=condition=available --timeout=300s deployment/tigera-operator -n tigera-operator

# 5. Create Calico Installation resource with correct CIDR
cat <<EOF | k apply -f -
apiVersion: operator.tigera.io/v1
kind: Installation
metadata:
  name: default
spec:
  calicoNetwork:
    ipPools:
    - blockSize: 26
      cidr: 10.244.0.0/16 # Must match your cluster CIDR
      encapsulation: VXLANCrossSubnet
      natOutgoing: Enabled
      nodeSelector: all()
  variant: Calico
  pointerSize: Felix
EOF

# 6. User custom resource (if needed)
# Usually not needed unless Tigera docs specify

# 7. Wait for Calico to be ready
k wait --for=condition=available --timeout=300s deployment -n calico-system -l k8s-app=calico-node

# 8. Check installation
k get pods -n calico-system
# Should show calico-node, calico-kube-controllers, calico-typha

# 9. Test networking
k run test1 --image=busybox:1.36 -- sleep 3600
k run test2 --image=busybox:1.36 -- sleep 3600

# Get IPs
POD1=$(k get pod test1 -o jsonpath='{.status.podIP}')
POD2=$(k get pod test2 -o jsonpath='{.status.podIP}')

# Test connectivity
k exec deploy/test1 -- ping -c 1 $POD2
# Should succeed
```

Reference: <https://docs.tigera.io/calico/latest/getting-started/kubernetes/self-managed-onprem/onpremises>

Exercise 28 — Complex NetworkPolicy (Multi-Namespace, Traffic Control)

1. Create 3 namespaces: `frontend`, `backend`, `database`
2. Deploy apps: - Frontend (nginx): labels `tier=frontend` - Backend (busybox): labels `tier=backend` - Database (redis): labels `tier=database`
3. Create NetworkPolicies: - Allow frontend → backend (only port 8080) - Allow backend → database (only port 6379) - Deny all other traffic initially (prefer-deny ingress) - Allow DNS egress (UDP 53) to kube-dns for all pods
4. **Explicitly test connectivity** to confirm policies are enforced: - Frontend CAN reach backend on 8080 only - Frontend CANNOT reach database - Backend CAN reach database on 6379 - Backend CANNOT reach frontend
5. Verify DNS still works from all namespaces (tests egress rules)
6. If policies don't enforce, verify CNI supports NetworkPolicy

Answer

```

# Create namespaces
k create ns frontend backend database

# Label namespaces for cross-ns selection
k label ns frontend name=frontend
k label ns backend name=backend
k label ns database name=database

# Deploy frontend
k run frontend-web -n frontend --image=nginx:1.27 --labels=tier=frontend

# Deploy backend
k run backend-api -n backend --image=busybox:1.36 --command sleep 3600 --labels=tier=backend

# Deploy database
k run database-redis -n database --image=busybox:1.36 --command sleep 3600 --labels=tier=database

# Create services
k expose pod frontend-web -n frontend --port=80 --type=ClusterIP
k expose pod backend-api -n backend --port=8080 --type=ClusterIP
k expose pod database-redis -n database --port=6379 --type=ClusterIP

# Allow frontend → backend
cat <<EOF | k apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-to-backend
  namespace: backend
spec:
  podSelector:
    matchLabels:
      tier: backend
  policyTypes:
  - Ingress
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          name: frontend
    ports:
    - protocol: TCP
      port: 8080
EOF

# Allow backend → database
cat <<EOF | k apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-backend-to-database
  namespace: database
spec:
  podSelector:
    matchLabels:
      tier: database
  policyTypes:
  - Ingress
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          name: backend
    ports:
    - protocol: TCP
      port: 6379
EOF

# Allow DNS egress from all namespaces
for ns in frontend backend database; do
cat <<EOF | k apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy

```

```

metadata:
  name: allow-dns-egress
  namespace: $ns
spec:
  podSelector: {}
  policyTypes:
  - Egress
  egress:
  - to: []
    ports:
    - protocol: UDP
      port: 53
EOF
done

# Test connectivity
# Frontend → Backend (should work)
k exec -it $(k get po -n frontend -o name | head -1) -n frontend -- curl -m 2 http://backend-
api.backend.svc.cluster.local:8080

# Frontend → Database (should fail)
k exec -it $(k get po -n frontend -o name | head -1) -n frontend -- curl -m 2 http://database-
redis.database.svc.cluster.local:6379

```

Exercise 29 — Troubleshoot Broken Cluster (Incorrect etcd Endpoint)

1. Cluster is broken: API server won't start
2. Check kube-apiserver pod logs
3. Find the error: incorrect etcd endpoint IP or port
4. SSH into control plane node
5. Edit `/etc/kubernetes/manifests/kube-apiserver.yaml`
6. Correct the etcd endpoint in `--etcd-servers=` flag
7. Verify IP/port matches `/etc/kubernetes/manifests/etcd.yaml`
8. Save and wait for API server to restart
9. Verify cluster is healthy: - `k get nodes` works - `k get pods -A` works - API server is Running

Answer

```
# 1. Check if API server is running (it may be stuck/restarting)
k get pod -n kube-system -l component=kube-apiserver

# 2. If accessible, check logs
k logs -n kube-system kube-apiserver-<node> --tail=50
# Look for error like: "dial tcp 10.0.0.5:2379: connection refused"

# 3. SSH into control plane node
ssh <control-plane-ip>

# 4. Check current etcd endpoint in API server manifest
grep "etcd-servers" /etc/kubernetes/manifests/kube-apiserver.yaml
# Example: --etcd-servers=https://10.0.0.5:2379

# 5. Get CORRECT endpoint from etcd manifest
grep "listen-client-urls" /etc/kubernetes/manifests/etcd.yaml
# Example: --listen-client-urls=https://127.0.0.1:2379,https://10.0.0.10:2379

# 6. The correct endpoint should be one from etcd's listen URLs
# Usually: https://127.0.0.1:2379 (if etcd on same node)
# Or: https://<etcd-ip>:2379 (if separate etcd node)

# 7. Edit and fix
sudo sed -i 's|--etcd-servers=https://10.0.0.5:2379|--etcd-servers=https://127.0.0.1:2379|' /etc/kubernetes/manifests/kube-apiserver.yaml

# Or manually edit
sudo vi /etc/kubernetes/manifests/kube-apiserver.yaml
# Find and correct the etcd-servers line

# 8. Save (vi: Esc, :wq)
# kubelet will auto-detect change and restart pod

# 9. Verify API server comes up (may take 10-15 seconds)
k get pod -n kube-system kube-apiserver-<node-name>
# STATUS should change from CrashLoopBackOff to Running

# 10. Test cluster is working
k get nodes
k version
```

Exercise 30 — TLS Configuration Update (Cipher Support)

1. A service currently supports only TLS 1.3
2. Add support for TLS 1.2 (make both available)
3. Update configuration file or ConfigMap: - Add TLS 1.2 to supported protocols - Keep TLS 1.3 enabled
4. ConfigMap is mounted in Deployment as env var or config file
5. Restart Deployment to apply changes
6. Verify service accepts TLS 1.2 connections: - Use `openssl s_client -connect <service> -tls1_2` - Or curl with explicit TLS 1.2
7. Verify TLS 1.3 still works

Answer

```
# 1. Find the service and ConfigMap
k get svc --all-namespaces
k get cm --all-namespaces

# 2. Check current TLS config
k get cm <name> -o yaml

# 3. Edit ConfigMap to add TLS 1.2
k edit cm <name>
# Find line like: tls_min_version: "1.3"
# Change to: supported_versions: ["1.2", "1.3"]
# Or if format is different, add TLS 1.2

# 4. Check if there's an env var that sets this
k describe deployment <deployment>
# Look for "Environment:" section

# 5. If config is in file mount, edit ConfigMap
# Then restart deployment

k rollout restart deployment/<deployment>

# 6. Wait for pod to be ready
k rollout status deployment/<deployment>

# 7. Portforward to test (if service is internal)
k port-forward svc/<service> <local-port>:<remote-port> &

# 8. Test TLS 1.2 connection
echo | openssl s_client -connect localhost:<port> -tls1_2 2>/dev/null | grep "Protocol"
# Should show: Protocol : TLSv1.2

# 9. Test TLS 1.3 still works
echo | openssl s_client -connect localhost:<port> -tls1_3 2>/dev/null | grep "Protocol"
# Should show: Protocol : TLSv1.3

# 10. Or use curl
curl -k --tlsv1.2 https://localhost:<port>
curl -k --tlsv1.3 https://localhost:<port>
```

Exercise 31 — Argo CD Setup (GitOps Deployment)

1. Install Argo CD in cluster: - Create `argocd` namespace - Apply Argo CD manifests (WITHOUT CRDs if specified)
2. Generate Kubernetes manifest template: - Define an Application resource - Point to a Git repository - Specify deployment target namespace - Do NOT install CRDs — let Argo CD install them
3. Save generated Application manifest to file
4. Verify Argo CD server is Running
5. Apply the Application manifest
6. Verify application deployment syncs
7. Create a port-forward to Argo CD UI (optional testing)

Answer

```
# 1. Create Argo CD namespace
k create namespace argocd

# 2. Install Argo CD (check https://argo-cd.readthedocs.io/en/stable/getting_started/)
# Use specific version, e.g., v2.10.3
k apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/v2.10.3/manifests/install.yaml

# Note: Don't apply -f manifests/crds.yaml separately

# 3. Wait for Argo CD to be ready
k wait --for=condition=available --timeout=300s deployment/argocd-server -n argocd

# 4. Create an Application manifest (GitOps)
cat <<EOF > application.yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: my-app
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/argoproj/argocd-example-apps.git
    targetRevision: HEAD
    path: guestbook
  destination:
    server: https://kubernetes.default.svc
    namespace: default
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
EOF

# 5. Apply the Application
k apply -f application.yaml

# 6. Watch sync status
k get application -n argocd -w

# 7. Check deployment status
k get pod -n default

# 8. Verify resources created by Argo CD
k get all -n default

# 9. Access Argo CD UI (optional)
k port-forward -n argocd svc/argocd-server 8080:443
# Navigate to https://localhost:8080
# Default password: admin / (stored in secret)
# Get password: k get secret argocd-initial-admin-secret -n argocd -o jsonpath='{.data.password}' | base64 -d

# 10. View application my-app sync status
k get application my-app -n argocd -o jsonpath='{.status.sync.status}'
# Output: Synced
```

References: - Argo CD Docs: <https://argo-cd.readthedocs.io/> - Application CRD: <https://argo-cd.readthedocs.io/en/stable/declarative-setup/>